LUNG DISEASE PREDICTION: DEEP LEARNING-DRIVEN PHONOPNEUMOGRAPHIC ANALYSIS FOR PULMONARY DISEASE RECOGNITION USING DFT AND MELSPECTROGRAMS

A project report submitted in partial fulfilment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Presented By

ARIPAKA LALITHAMBA **226K1A4205**

Under the Esteemed Guidance of

Mr. M. V. V. Chowdary, M. Tech.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

IDEALINSTITUTE OF TECHNOLOGY

Approved by AICTE, Accredited by NAAC with A+, Affiliated to JNTUK,

Kakinada Vidyut Nagar, Kakinada-533003, Kakinada Dist., Andhra Pradesh.

2022-2026

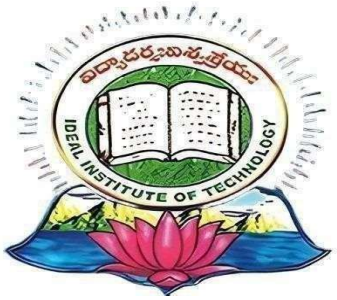
IDEALINSTITUTE OF TECHNOLOGY

Approved by AICTE, Accredited by NAAC with A+, Affiliated to JNTUK, Kakinada

Vidyut Nagar, Kakinada-533003, Kakinada Dist., Andhra Pradesh.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)



CERTIFICATE

This is to certify that the project work entitled, **"LUNG DISEASE PREDICTION"** is a Bonafide work of **ARIPAKA LALITHAMBA (226K1A4205),** submitted to the department of **Computer Science and Engineering(Artificial Intelligence & Machine Learning)**, in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING (Artificial Intelligence & Machine Learning)** from IDEAL INSTITUTE OF TECHNOLOGY, Kakinada.

Signature of the Project Guide

Mr. M. V. V. Chowdary, M.Tech.

Assistant Professor

Signature of the HOD

Mr. M.S.R.S. Prasad, M.Tech., [Ph.d]

Head of the Department

External Examiner

DECLARATION

We hereby declare that the project report entitled, **"LUNG DISEASE PREDICTION"** has been carried out and contents have been submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING (Artificial Intelligence & Machine Learning), IDEAL INSTITUTE OF TECHNOLOGY**, Affiliated to JNTUK, Kakinada. We assure that this project report has not been submitted to any other University or College.

NAME	RGD.NO	SIGNATURE
ARIPAKA LALITHAMBA	**226K1A4205**	

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratefulness and deep regards to our Chairman **Dr P. Krishna Rao**, for encouraging us and providing us with suitable facilities for successful completion of the project.

I express my deep sense of heartfelt gratitude and regard to our Secretary & Correspondent **Sri P. S. S. Kiran,** for his incessant support and blessings given from time to time shall carry us a long way in the journey of life on which we are about to embark.

I am truly indebted and thankful to our Director **Sri G.V. Krishnam Raju,** for his continuous supervision towards the completion of our course work.

I am extremely grateful to our Administrative Officer **Sri G.V.K. Varma,** for his continuous supervision towards the completion of our course work.

I express my sincere gratitude to our Principal **Dr T. Srikanth,** for his constant encouragement and timely suggestions during academic period.

I place a deep sense of thanks to our Computer Science and Engineering department HOD **Sri M.S.R.S.Prasad,** who has been constant source of inspiration during the preparation of this project work.

I am indebted to our project guide **Mr. M. V. V. Chowdary,** for giving valuable suggestions for the advancement of the Project.

I am obliged to faculty of **COMPUTER SCIENCE AND ENGINEERING** Department, for the valuable information provided by them in their respective fields.

I am grateful for their cooperation during the period of our assignment.

I am very much thankful to the **Technical Supporting Staff** who helped me for the successful completion of the project.

ARIPAKA LALITHAMBA **226K1A4205**

IDEAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada &
Accredited by NAAC with 'A+' Grade) Vidyut Nagar, KAKINADA-533003(A.P.)

COLLEGE VISION:

- To emerge as a premier institution for technical education in the country through academic excellence and to be recognized as a center for excellence in research & development, catering to the needs of our country.

COLLEGE MISSION:

- To establish a robust institution that consistently maintains state-of-the-art infrastructure facilities.
- To create a cohesive, world-class team of faculty dedicated to innovative and experimental teaching–learning processes.
- To enhance industry interaction and facilitate the delivery of technical education and research tailored to meet the specific needs of the country.

IDEAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada &
Accredited by NAAC with 'A+' Grade) Vidyut Nagar, KAKINADA-533003(A.P.)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT VISION:

- To build a strong teaching-learning base with a flair for innovation and research that responds to the dynamic needs of the software industry and the society with good ethical practices.

DEPARTMENT MISSION:

- To provide strong foundation both in theory and applications of Computer Science & Engineering, to solve real-world problems.
- To empower students with state-of-art knowledge and up to date technological skills, making them globally competent.
- Inculcating professional behavior, strong ethical values, innovative research capabilities and leadership abilities.

IDEAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada &
Accredited by NAAC with 'A+' Grade), Vidyut Nagar, KAKINADA-533003(A.P.)

Bachelor of Technology Program Outcomes (POs)

Engineering Graduates will be able to:

PO1	Engineering knowledge:	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis:	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/ developmentof solutions:	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations complex problems:	Use research-based knowledge and research methods including design of experiment analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage:	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
PO6	Theengineer and society:	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and sustainability:	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics:	Apply ethical principles and commit to professional ethics and Responsibilities and norms of the engineering practice.
PO9	Individual and team work:	Function effectively as an individual, and as a member or leader in Diverse teams, and in multidisciplinary settings.
PO10	Communication:	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project managementand finance:	Demonstrate knowledge and understanding of the engineering and Management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning:	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

IDEAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada &
Accredited by NAAC with 'A+' Grade), Vidyut Nagar, KAKINADA-533003(A.P.)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ProgramSpecific Outcomes (PSOs)

PSO1: The ability to comprehend, analyze, design and incorporate application-based solutions.

PSO2: To facilitate oneself with the capability to solve complex projects using updated tools and technologies.

Index

TOPIC	PAGE NO
List of Figures	i
List of Tables	ii
List of Abbreviations	iii
Abstract	iv
1. INTRODUCTION	**[01-5]**
1.1 About the project	1
1.2 Modules of The Project	5
2. LITERATURE REVIEW	**[6-10]**
2.1 Reviews	7
2.1.1 Review-1	7
2.2.2 Review-2	8
2.2.3 Review-3	9
2.2.4 Review-4	10
3. SYSTEM REQUIREMENTS SPECIFICATIONS	**[11-38]**
3.1 Existing System	12
3.2 System Requirements Specifications	17
3.2.1 Front End and Back End Languages	17
3.2.2 User Requirements	32
3.2.3 Hardware Requirements	32
3.2.4 Software Requirements	33
3.3 Feasibility Study	35
3.3.1 Technical Feasibility	36
3.3.2 Operational Feasibility	37
3.3.3 Economical Feasibility	38
4. SYSTEM DESIGN	**[39-56]**
4.1 System Architecture	40
4.1.1 Flow Chart Diagram	44
4.2 UML Diagrams	46-56
4.2.1 Use Case Diagram	49
4.2.2 Class Diagram	50
4.2.3 Activity Diagram	51
4.2.4 Sequence Diagram	52
4.2.5 Component Diagram	55
4.2.6 Deployment Diagram	56
5. SAMPLE CODE	**[57-81]**

6. IMPLEMANTATION OF PROJECT **[82-89]**
6.1 Introduction 83
6.1.1 Setting up the Development Environment 85
6.1.2 Installation of Python 86
6.1.3 Installation of Required Libraries 87
6.1.4 Project Folder Structure 88
6.1.5 Configuration of Flask Web Framework 89
6.1.6 Setting up Required Directories 89
6.1.7 Running the Application 89
6.2 Testing the Setup 90
6.3 Audio Processing Implementation **[91-96]**
6.3.1 Loading The Audio File 92
6.3.2 Resampling The Audio Signal 92
6.3.3 Removing Silence From The Audio 92
6.3.4 Normalizing The Audio Signal 93
6.3.5 Feature Extraction Using Mel Spectrogram 93
6.3.6 Waveform Visualization 94
6.3.7 Spectrogram Visualization 95
6.3.8 Importance of Audio Processing 95
6.4 Explanation of Modules 96
6.4.1 Preprocessing Modules **[96-104]**
6.4.2 Data Augmentation Module 97
6.4.3 Time-Frequency Representation Module 99
6.4.4 CNN-LSTM Classification Module 101
102

7.TESTING **[105-112]**
7.1 Testing 106
7.2 Types of Testing 106
7.3 Test Cases **[109-112]**
7.3.1 Test Case 1 109
7.3.2 Test Case 2 109
7.3.3 Test Case 3 110
7.3.4 Test Case 4 110
7.3.5 Test Case 5 111
7.3.6 Test Case 6 111
7.3.7 Test Case 7 112
7.3.8 Test Case 8 112

8.RESULTS AND DISCUSSION **[113-122]**
8.1 Results 114
9.CONCLUSIONS AND FUTURE SCOPE **[123-125]**
9.1 Conclusion 124
9.2 Future Scope 125
10. BIBLIOGRAPHY **[126-127]**

LIST OF FIGURES

Figure Number	Figure Name	Page Number
1	System Architecture	40
2	Flow Chart Diagram	45
3	Use Case Diagram	49
4	Sequence Diagram	54
5	User Login Page	114
6	Home Page	114
7	Patient Data Input Page	115
8	Prediction Result Page	117
9	Lung Disease Prediction Graph	118
10	Model Accuracy Comparison	119
11	Final prediction Output Page	122

LIST OF TABLES

Table	Table Name	Page Number
3.2.2	Hardware Requirements	32
3.2.3	Software Requirements	33
7.3.1	Test Case 1	109
7.3.2	Test Case 2	109
7.3.3	Test Case 3	110
7.3.4	Test Case 4	110
7.3.5	Test Case 5	111
7.3.6	Test Case 6	111
7.3.7	Test Case 7	112
7.3.8	Test Case 8	112

LIST OF ABBREVIATIONS

AI	:	Artificial Intelligence
ML	:	Machine Learning
DL	:	Deep Learning
CNN	:	Convolutional Neural Network
LSTM	:	Long Short-Term Memory
MFCC	:	Mel Frequency Cepstral Coefficients
DFT	:	Discrete Fourier Transform
GUI	:	Graphical User Interface
UI	:	User Interface
HTML	:	Hyper Text Markup Language
CSS	:	Cascading Style Sheets
JS	:	JavaScript
API	:	Application Programming Interface
DBMS	:	Database Management System
SQL	:	Structured Query Language
IDE	:	Integrated Development Environment
OS	:	Operating System
CPU	:	Central Processing Unit
RAM	:	Random Access Memory
URL	:	Uniform Resource Locator
PDF	:	Portable Document Format
COPD	:	Chronic Obstructive Pulmonary Disease
URTI	:	Upper Respiratory Tract Infection

ABSTRACT

Respiratory diseases are the world's largest cause of morality. Classifying respiratory diseases using lung sound analysis helps in early detection of diseases. Users submit lung sound recordings, which undergo a series of steps for effective processing. The initial resampling at 4KHz standardizes the audio data, followed by temporal snippet generation to identify reference points. Noise removal through DFT-based baseline wander removal enhances data quality. The signal is then boosted using Amplitude normalization. Feature extraction involves log-scaled Mel spectrogram analysis to capture disease-specific patterns.

The processed sound is fed into a CNN- LSTM model trained to predict six respiratory diseases, including Bronchiolitis, Bronchitis, Pneumonia, LRTI, Asthma, and COPD. Users are also provided with precautionary measures to reduce the impact of respiratory conditions. The website aims to contribute to early detection and management through accurate disease classification based on sound patterns.

Feature extraction is carried out using log-scaled Mel spectrograms, which effectively capture the time-frequency characteristics of lung sounds and highlight disease-specific patterns. The processed features are then fed into a hybrid deep learning model combining Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks. This model is trained to accurately classify multiple respiratory diseases, including Bronchiolitis, Bronchitis, Pneumonia, Lower Respiratory Tract Infections (LRTI), Asthma, and Chronic Obstructive Pulmonary Disease (COPD), by learning both spatial and temporal dependencies in the data.

Chapter-1
INTRODUCTION

1. INTRODUCTION

1.1 About the project

Lung Disease Prediction using Deep Learning is a web-based healthcare application designed to detect possible lung diseases by analyzing respiratory sounds. Lung diseases such as asthma, pneumonia, bronchitis, and COPD are common health problems that require early diagnosis for effective treatment. In traditional methods, doctors use a stethoscope to listen to breathing sounds and identify abnormalities. However, this manual process may sometimes be time-consuming and depends on the experience of the doctor. This project aims to develop an automated system that uses deep learning techniques to analyze breathing sounds and predict lung diseases accurately. The system allows users to upload or record respiratory sound through a website, making the diagnosis process easier and faster.

The proposed system uses audio processing and deep learning algorithms to analyze the uploaded sound. The audio file is processed using libraries such as Librosa to extract important sound features like MFCC. These features are then given to a trained deep learning model, such as a Convolutional Neural Network (CNN). The backend of the system is developed using Python and Flask, while the frontend uses HTML, CSS, and JavaScript to create a user-friendly interface. The system displays the prediction result on the website, helping users understand possible lung conditions. This project demonstrates how artificial intelligence and web technologies can be combined to support early detection and improve healthcare services.

Lung Disease Prediction using Deep Learning is a web-based healthcare application designed to detect possible lung diseases by analyzing respiratory sounds. Lung diseases such as asthma, pneumonia, bronchitis, and Chronic Obstructive Pulmonary Disease (COPD) are common health problems that affect millions of people around the world. Early diagnosis of these diseases is very important because it allows doctors to begin treatment at the right time and reduce serious health complications.

In traditional healthcare systems, doctors diagnose lung diseases by listening to breathing sounds using a stethoscope. They carefully analyze abnormal sounds such as wheezing, crackles, or reduced airflow to identify respiratory problems. Although this method is widely used, it depends heavily on the experience and expertise of the doctor. In busy hospitals or rural healthcare centers, manual diagnosis may sometimes take more time and may not always provide highly accurate results.

The main objective of this project is to develop an intelligent system that can automatically analyze respiratory sounds and predict lung diseases using deep learning technology. The system allows users to upload or record breathing sounds through a web application. Once the sound is uploaded, the system processes the audio signal and extracts important features from the respiratory sound.

In this project, audio processing techniques are used to convert breathing sounds into meaningful data that can be analyzed by machine learning models. The audio files are processed using Python libraries such as Librosa, which helps in extracting important sound features like Mel Frequency Cepstral Coefficients (MFCC), spectral features, and other acoustic properties. These features represent the characteristics of the breathing sound and help the model understand patterns related to different lung diseases.

After feature extraction, the processed data is given to a deep learning model such as a Convolutional Neural Network (CNN). CNN models are widely used in pattern recognition tasks because they can automatically learn complex patterns from data. The model is trained using a dataset containing respiratory sounds from both healthy individuals and patients with lung diseases. During training, the model learns to differentiate between normal and abnormal breathing patterns.

The backend of the application is developed using Python and Flask, which handles data processing, model execution, and prediction generation. The frontend of the system is designed using HTML, CSS, and JavaScript to provide a simple and user-friendly interface.

Users can easily upload an audio file or record breathing sounds through the website. Once the sound is processed, the system displays the predicted result along with possible lung conditions.

This system provides several advantages in healthcare applications. It helps in early detection of lung diseases, reduces the workload of medical professionals, and supports doctors in making better clinical decisions. The system can also be useful in remote healthcare environments where access to specialized doctors is limited. Patients can upload their respiratory sounds and receive a preliminary prediction that encourages them to seek medical advice if needed.

Furthermore, the integration of artificial intelligence, deep learning, and web technologies makes this project a modern healthcare solution. It demonstrates how advanced technologies can assist in improving disease diagnosis and healthcare efficiency. With further development, the system can be integrated with mobile applications, medical devices, and hospital databases to create a more powerful diagnostic tool.

In the future, the project can be enhanced by using larger medical datasets, improving the deep learning model accuracy, and integrating real-time respiratory monitoring systems. Advanced models such as Recurrent Neural Networks (RNN) or hybrid deep learning models can also be used to improve prediction performance. The system may also include additional features such as patient health history analysis, automated medical reports, and doctor consultation support.

1.2 Modules of the project

Administrator Login

- **USER AUDIO UPLOAD**
    - Upload respiratory sound file
    - Record breathing sound
    - Validate audio format
    - Send audio to server for processing

- **AUDIO PREPROCESSING**
    - Receive uploaded sound file
    - Remove noise from audio
    - Convert audio into spectrogram
    - Extract MFCC features

- **DEEP LEARNING PREDICTION**
    - Load trained CNN model
    - Analyze extracted audio features
    - Classify respiratory sound patterns
    - Predict possible lung disease

Chapter-2
LITERATURE REVIEW

2. LITERATURE REVIEW

2.1 Reviews

2.1.1 Review – 1

Traditional Methods For Lung Disease Diagnosis:

In the early stages of medical diagnosis, lung diseases were mainly identified through physical examination and clinical observation. Doctors used instruments such as stethoscopes to listen to respiratory sounds produced by patients during breathing. These sounds, including wheezing, crackles, and abnormal breathing patterns, helped physicians determine whether a patient had any respiratory illness. While this method has been widely used for many years, it largely depends on the experience and expertise of the medical professional. The interpretation of lung sounds may vary from one doctor to another, which can sometimes lead to inaccurate or delayed diagnosis.

Manual analysis of respiratory sounds can be time-consuming and may not always detect subtle patterns present in the sound signals. Additionally, the lack of specialized medical professionals in rural or remote areas makes it difficult for patients to receive timely diagnosis. These challenges have encouraged researchers to explore technological solutions that can assist doctors in identifying lung diseases more accurately and efficiently.

Drawbacks:

- Traditional diagnosis methods rely heavily on the doctor's experience and listening skills.
- Manual examination can sometimes lead to inaccurate or inconsistent results.

2.1.2 Review-2

Machine Learning Approaches For Lung Disease Detection:

With the advancement of computer technology, machine learning techniques have been introduced to assist in the diagnosis of respiratory diseases. Machine learning algorithms can analyze medical data and identify patterns that may not be easily recognized by humans. In lung disease detection, researchers have used machine learning models to analyze respiratory sound recordings collected from patients. These sounds are processed using signal processing techniques to extract important features such as frequency patterns and amplitude variations.

After extracting these features, machine learning algorithms such as Support Vector Machines (SVM), Decision Trees, and Random Forest classifiers are used to classify the sounds into different categories such as normal breathing, wheezing, or crackles. These approaches have shown promising results in improving diagnostic accuracy and reducing the workload of healthcare professionals.

However, traditional machine learning methods require manual feature extraction and careful selection of parameters, which can make the system complex and less efficient when dealing with large datasets.

Drawbacks:

- Machine learning models require manual feature extraction and preprocessing.
- The performance of the system depends heavily on the quality of the extracted features.

2.1.3 Review - 3

Deep Learning Techniques For Respiratory Sound Classification:

Deep learning has emerged as a powerful technique for analyzing complex medical data such as images, speech, and audio signals. In recent years, deep learning models have been applied to respiratory sound analysis for lung disease detection. Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) are commonly used architectures that can automatically learn important patterns from raw audio data. Unlike traditional machine learning approaches, deep learning models do not require manual feature engineering because they can automatically extract relevant features during the training process.

In lung disease prediction systems, respiratory sound signals are often converted into spectrograms or Mel Frequency Cepstral Coefficients (MFCC) before being fed into the deep learning model. These representations help the model understand the frequency and time characteristics of breathing sounds.

Deep learning models have demonstrated higher accuracy in identifying abnormal respiratory sounds and classifying different lung diseases. As a result, they have become a popular approach for developing intelligent healthcare systems.

.

Drawbacks:

- Deep learning models require a large amount of labeled data for effective training.
- Training deep learning models can be computationally expensive and time-consuming.

2.1.4 Review - 4

Web-Based Healthcare Systems for Disease Prediction:

With the rapid growth of internet technologies, web-based healthcare systems have become an effective way to provide medical services remotely. These systems allow patients to access healthcare tools and diagnostic services through web applications without visiting hospitals frequently. In lung disease prediction systems, web technologies are used to create user-friendly interfaces where patients can upload respiratory sound recordings for analysis. The uploaded audio files are processed by backend systems that use machine learning or deep learning models to predict possible diseases.

Web-based systems also enable easy storage and management of patient data while providing quick access to diagnostic results. Technologies such as HTML, CSS, JavaScript, and Python frameworks like Flask are commonly used to develop these applications. By integrating deep learning models with web applications, healthcare services can be made more accessible and efficient. Such systems can assist doctors in making faster decisions and provide early detection of lung diseases for better treatment outcomes.

Drawbacks:

- Web-based healthcare systems require reliable internet connectivity for proper functioning.
- Ensuring the privacy and security of patient data is a major challenge in online medical systems.

Chapter-3

SYSTEM REQUIREMENTS SPECIFICATION

3. SYSTEM REQUIREMENTS SPECIFICATION

3.1 Existing System:

In the current healthcare system, the diagnosis of lung diseases is mainly performed through traditional medical procedures and manual analysis by doctors. Physicians study patient symptoms, medical history, laboratory reports, and medical imaging techniques such as Chest X- rays, CT scans, and pulmonary function tests to identify lung-related diseases. While these methods are effective, they often require significant time, experience, and careful examination.

In many hospitals and medical institutions, patient data and diagnostic reports are often stored either manually or in isolated digital systems. This creates difficulties when doctors need to quickly access or analyze patient information. Managing patient records manually increases the workload of medical staff and also increases the chances of data loss, duplication, or errors.

Another limitation of the existing system is the absence of automated disease prediction tools. Doctors must carefully interpret medical images and test reports to identify diseases such as pneumonia, tuberculosis, lung cancer, asthma, or chronic bronchitis. This process can sometimes be complex and time-consuming, especially when there are many patients to diagnose.

Furthermore, the traditional system lacks integration with advanced technologies such as Artificial Intelligence (AI) and Machine Learning (ML). These technologies have the ability to analyze large medical datasets and identify patterns that may not always be easily visible to the human eye. Without such tools, early detection of lung diseases becomes more difficult.

In addition, the existing systems often require patients to wait longer for diagnosis and results. Delays in identifying lung diseases can negatively impact patient treatment and recovery. Early detection is very important for effective treatment, but the traditional system does not always support quick and efficient disease prediction.

In the traditional healthcare system, lung disease diagnosis is performed manually by doctors using different medical tools and reports. The diagnosis process requires collecting and analyzing several types of patient information and medical records. Doctors must carefully examine these details before making a final diagnosis.

The diagnosis process usually involves the use of medical reports and analysis tools such as medical imaging software, laboratory reports, and hospital record systems. Doctors also need to review multiple pieces of patient information to understand the patient's health condition accurately.

The diagnosis requires many details such as:

- Patient's Name
- Patient ID or Registration Number
- Age and Gender
- Medical History
- Symptoms (such as cough, chest pain, breathing difficulty, fever)
- Chest X-ray or CT scan images
- Laboratory test reports
- Doctor's observations and notes
- Previous treatment records
- Hospital or diagnostic center details

Disadvantages:

- **High Workload for Medical Staff**

Doctors and hospital staff need to manage large amounts of patient data, medical reports, and test results. This increases their workload and may slow down the diagnosis process.

- **Manual Diagnosis is Time-Consuming**

In the existing healthcare system, diagnosing lung diseases often requires multiple medical tests and detailed examination by doctors. This process can take a long time, especially when there are many patients waiting for diagnosis.

- **Dependence on Doctor Experience**

The accuracy of diagnosis depends largely on the experience and expertise of the doctor. Different doctors may interpret the same medical reports or respiratory sounds differently, which may lead to variations in diagnosis.

- **Limited Use of Automated Tools**

Most traditional healthcare systems do not use advanced technologies such as Artificial Intelligence or Machine Learning for disease prediction. As a result, the diagnosis process relies mainly on manual interpretation of medical reports and imaging results

- **Difficulty in Early Detection**

Early detection of lung diseases is very important for effective treatment. However, in the traditional system, diseases are often identified only after symptoms become severe or when detailed tests are performed.

- **High Workload for Medical Staff**

Doctors and hospital staff need to manage large amounts of patient data, medical reports, and test results. This increases their workload and may slow down the diagnosis process.

- **Limited Accessibility in Rural Areas**

In rural or remote areas, access to specialized doctors and medical facilities is limited. Patients may need to travel long distances to get proper diagnosis and treatment.

Proposed System:

The proposed system is a Lung Disease Prediction System designed to assist in the early detection and prediction of lung-related diseases using modern technologies such as machine learning, data analysis, and web-based applications. This system aims to overcome the limitations of traditional diagnosis methods, which often require multiple hospital visits, expensive tests, and a long waiting time for medical reports.

The system provides an automated platform where users can input medical parameters and symptoms related to lung health. These inputs may include factors such as age, smoking habits, air pollution exposure, breathing problems, coughing frequency, chest pain, fatigue, and other respiratory symptoms. Once the user enters the required details, the system processes the data using a trained machine learning model to predict the possibility of lung disease.

The prediction model is developed using historical medical datasets that contain information about patients who have previously been diagnosed with lung diseases. By analyzing patterns in this data, the model learns to identify relationships between symptoms and specific lung conditions. As a result, the system can provide a probability-based prediction indicating whether a person is at risk of developing a lung disease or not.

This proposed system can be implemented as a web-based application, making it accessible from anywhere at any time through the internet. Doctors, healthcare workers, and even patients can use this system to get a quick preliminary analysis of lung health. The system does not replace professional medical diagnosis but acts as a support tool that helps in early screening and awareness.

Another important feature of the system is the storage and management of patient data. The system maintains a database where user inputs and prediction results are stored securely. This data can later be used for further analysis, research, and improvement of the prediction model.

Advantages:

• Early Detection of Lung Diseases

One of the most important advantages of the Lung Disease Prediction System is its ability to help in the early detection of lung-related diseases. Many lung diseases such as asthma, chronic bronchitis, pneumonia, and lung cancer develop slowly and may not show clear symptoms in the early stages. If these diseases are detected late, treatment becomes more difficult and costly.

The proposed system analyzes the symptoms and health parameters entered by the user and predicts the possibility of lung disease at an early stage. Early prediction can encourage patients to consult doctors sooner and begin treatment earlier, which can significantly improve the chances of recovery and reduce health risks.

- **Faster Prediction and Diagnosis Support**

Traditional medical diagnosis often involves multiple medical tests such as X-rays, CT scans, or laboratory reports, which can take several days to complete. The Lung Disease Prediction System provides quick and automated predictions using machine learning algorithms. Once the user enters the required information into the system, the algorithm processes the data and generates the prediction within seconds. This reduces waiting time and allows both patients and healthcare providers to obtain quick insights into a person's lung health condition.

3.2 System Requirements Specification:

3.2.1 Front End and Back End Languages

Front End:

1) HTML
2) CSS
3) JAVASCRIPT

Back End:

1) PYTHON
2) FLASK

Deep Learning:

1) TENSORFLOW

Audio Processing:

1) LIBROSA

HTML

Hypertext Markup Language (HTML) is the standard markup language for creating web pages and web applications with Cascading Style Sheets (CSS) and Java Script, it forms a triad of corner stone technologies for the World Wide Web. Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document. HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page.

HTML allows developers to create structured documents by using tags and elements. These tags define different parts of a webpage such as headings, paragraphs, links, tables, images, and multimedia content. HTML tags are usually written in pairs consisting of an opening tag and a closing tag. The opening tag indicates the beginning of an element and the closing tag indicates the end of that element.

For example, the <p> tag is used to define a paragraph, <h1> to <h6> tags are used for headings, and the <a> tag is used to create hyperlinks that connect one webpage to another.

HTML also supports embedding images, audio, and video elements, making web pages more interactive and visually appealing. Images can be displayed using the <img> tag, while videos and audio files can be embedded using the <video> and <audio> tags. These multimedia elements enhance the user experience and make the content more engaging.

Another important feature of HTML is the ability to create forms. Forms are used to collect input from users, such as login information, feedback, or uploaded files. Form elements include text fields, radio buttons, checkboxes, dropdown menus, and submit buttons. These forms allow websites to interact with users and send data to servers for processing.

HTML works closely with CSS and JavaScript. While HTML provides the structure of the webpage, CSS is used to style the webpage by adding colors, layouts, fonts, and animations.

HTML has evolved over time, and the latest version is HTML5. HTML5 introduced many new features such as semantic elements (<header>, <footer>, <section>, <article>), improved multimedia support, and better compatibility with modern web technologies. These features help developers create more efficient, accessible, and responsive web applications.

In the context of the Lung Disease Prediction project, HTML plays an important role in designing the user interface of the web application. It is used to create the structure of the web pages where users can upload respiratory sound files or record breathing sounds.

The HTML pages provide buttons, input fields, and display areas where the prediction results are shown to the user. By combining HTML with CSS and JavaScript, the system provides a simple and user- friendly interface that allows users to interact with the application easily.

Overall, HTML is a fundamental technology for web development and serves as the backbone of web applications. It enables developers to design structured, interactive, and accessible web pages that can be viewed across different browsers and devices.

CSS

CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript. CSS is designed to enable the separation of presentation and content, including layout, colors, and fonts.

This separation can improve content accessibility, provide more flexibility and control in the specification of presentation characteristics, enable multiple web pages to share formatting by specifying the relevant CSS in a separate CSS file, and reduce complexity and repetition in the structural content.

CSS allows web developers to design attractive and visually appealing web pages by applying styles to HTML elements. Using CSS, developers can control the layout, background colors, font styles, spacing, borders, and alignment of elements on a webpage. By separating the styling from the HTML structure, CSS makes it easier to maintain and update the design of multiple web pages at once.

CSS works through rules that consist of selectors and declarations. The selector indicates which HTML element will be styled, and the declaration defines the style to be applied. For example, developers can use CSS to change the color of text, adjust the size of images, or create different layouts for different sections of a webpage. This approach allows for consistent styling across an entire website.

One of the important features of CSS is its ability to create responsive web designs. Responsive design ensures that web pages adapt to different screen sizes and devices such as desktops, laptops, tablets, and mobile phones. CSS media queries allow developers to apply different styles depending on the screen size, making websites more accessible and user-friendly.

Another important advantage of CSS is that it improves the performance of web pages. Since CSS styles can be stored in external files and reused across multiple pages, it reduces the amount of repeated code and helps web pages load faster. This also makes it easier for developers to manage large websites with many pages.

In modern web development, CSS is widely used along with frameworks and libraries such as Bootstrap, Tailwind CSS, and Material UI. These frameworks provide pre-designed styles and components that help developers build professional-looking websites quickly and efficiently.

In the Lung Disease Prediction project, CSS plays a significant role in designing the visual appearance of the web application. It is used to style the webpage where users upload respiratory sound files and view prediction results.

CSS helps in designing buttons, forms, navigation menus, and result display sections in an organized and attractive way. With the help of CSS, the application provides a clean, responsive, and user-friendly interface that improves the overall user experience.

Overall, CSS is an essential technology for web development that enhances the visual presentation of web pages. By combining CSS with HTML and JavaScript, developers can create modern, interactive, and responsive web applications that work effectively across different devices and platforms.

JAVASCRIPT

JavaScript is a high-level programming language used to create interactive and dynamic web pages. It works together with HTML and CSS to enhance the functionality of websites. JavaScript allows developers to implement features such as form validation, audio file uploading, button actions, and real-time updates on the webpage. In web applications, JavaScript helps in handling user events like clicking buttons, selecting files, or submitting forms. It can also communicate with the server without refreshing the page, making the website faster and more responsive. In this project, JavaScript is used to manage the uploading of respiratory sound files and to send the data to the backend server for further processing and prediction.

JavaScript is widely used in modern web development because it enables websites to become more interactive and user-friendly. With JavaScript, developers can control how a webpage behaves when users interact with it. For example, JavaScript can display messages, validate user inputs before submitting forms, create animations, and dynamically update webpage content without reloading the entire page.

One of the key features of JavaScript is its ability to manipulate the Document Object Model (DOM). The DOM represents the structure of a web page as a tree of objects. Using JavaScript, developers can access and modify HTML elements in real time. This allows the webpage to change its content, style, or layout based on user actions.

Another important aspect of JavaScript is its support for frameworks and libraries that simplify the development process. Popular JavaScript libraries and frameworks include React, Angular, and Vue.js. These tools help developers build complex and scalable web applications more efficiently.

For example, when a user uploads an audio file, JavaScript can immediately display the file name or show a preview before sending it to the server. JavaScript also supports asynchronous communication with servers using technologies such as AJAX (Asynchronous JavaScript and XML) and the Fetch API. These technologies allow the browser to send and receive data from the server without reloading the webpage. This improves the speed and efficiency of web applications and provides a smoother user experience.

JavaScript is also capable of handling multimedia elements such as audio and video files. It can control the playback of audio, record sound from a microphone, and process user interactions related to multimedia content. This capability is particularly useful in applications that involve audio analysis or recording.

In the Lung Disease Prediction project, JavaScript plays an important role in improving the interaction between the user and the web application. It is used to handle the uploading of respiratory sound files, manage button actions, and send user input to the server for analysis. JavaScript also helps display messages or results on the webpage after the prediction is completed. This makes the system more interactive and convenient for users.

Additionally, JavaScript helps improve the overall user experience by providing instant feedback and smooth interaction with the web application. For example, when a user uploads an audio file, JavaScript can verify whether the file format is correct before sending it to the server. This prevents errors and ensures that only valid data is processed by the system.

Overall, JavaScript is an essential technology in modern web development. By combining JavaScript with HTML and CSS, developers can create powerful, interactive, and responsive web applications. In this project, JavaScript plays a crucial role in connecting the frontend interface with the backend processing system, ensuring that respiratory sound data is properly transmitted and the prediction results are displayed effectively to the user.

PYTHON

Python is a powerful, high-level programming language widely used for web development, data analysis, machine learning, and artificial intelligence. It is known for its simple syntax and easy readability, which makes it suitable for developing complex applications quickly. Python provides a large number of libraries and frameworks that help developers build efficient systems. In this project, Python is used as the backend programming language to process the respiratory sound data received from the website. It also helps in connecting the web application with the deep learning model and performing data processing tasks required for lung disease prediction.

Python is widely preferred by developers because of its flexibility and extensive community support. It supports multiple programming paradigms such as object-oriented programming, procedural programming, and functional programming. This makes Python a versatile language that can be used in various domains including scientific computing, automation, data science, and web development.

One of the major advantages of Python is the availability of powerful libraries and frameworks. Libraries such as NumPy and Pandas are used for numerical computing and data manipulation, while libraries such as Matplotlib and Seaborn are used for data visualization. In the field of machine learning and deep learning, libraries like TensorFlow, Keras, and PyTorch provide advanced tools for building and training intelligent models.

Python is also widely used for backend web development through frameworks such as Flask and Django. These frameworks allow developers to build web applications that can interact with databases, process user input, and communicate with machine learning models. Flask is a lightweight web framework that is commonly used for developing simple and scalable web applications, while Django provides a more comprehensive framework for building large and complex systems.

FLASK

Flask is a lightweight web framework written in Python that is used for developing web applications. It provides tools and libraries that make it easier to build web servers and connect the frontend with the backend. Flask allows developers to create routes, handle HTTP requests, and manage communication between the user interface and the server.

It is widely used in machine learning projects because it can easily integrate trained models into web applications. In this project, Flask acts as the bridge between the website and the deep learning model. It receives the uploaded respiratory sound from the user, processes the data using Python libraries, and sends the prediction result back to the website.

Flask is known for its simplicity and flexibility, which makes it an ideal choice for developing small to medium-scale web applications. Unlike some other web frameworks, Flask does not require a complex structure to start a project. Developers can quickly create a web application using minimal code. This simplicity makes Flask highly suitable for research-based applications and machine learning projects where rapid development and testing are required.

One of the key features of Flask is its routing system. Routing allows developers to define different URLs that perform specific functions in the web application. For example, one route may handle the home page of the website, while another route may handle the uploading of respiratory sound files. When a user interacts with the webpage, Flask receives the request, processes it, and sends an appropriate response back to the browser.

Flask also supports the use of templates to generate dynamic web pages. Templates allow developers to create web pages that can display different data depending on the user's request. The Jinja2 templating engine is commonly used with Flask to integrate HTML pages with Python code. This allows developers to display prediction results, messages, and other dynamic content directly on the webpage.

Another important feature of Flask is its ability to handle file uploads and form submissions. In many web applications, users need to upload files such as images, documents, or audio recordings. Flask provides built-in support for receiving uploaded files from the frontend and storing them on the server for further processing. In this project, this feature is used to receive respiratory sound recordings from users.

Flask can also be easily integrated with machine learning and deep learning models. Once a model is trained using libraries such as TensorFlow, Keras, or PyTorch, it can be loaded into a Flask application. When the user uploads data, Flask sends the data to the model, which performs the prediction and returns the result. This makes Flask a powerful tool for deploying artificial intelligence systems on the web.

Flask also supports integration with databases and other external services. If required, the application can store user data, prediction history, or medical records in a database system. This capability makes it possible to extend the project into a more advanced healthcare platform.

Overall, Flask plays a vital role in connecting different components of the Lung Disease Prediction system. It connects the frontend interface developed using HTML, CSS, and JavaScript with the backend processing system implemented in Python. By managing user requests, processing data, and delivering prediction results, Flask ensures that the entire application works smoothly and efficiently.

TENSORFLOW

TensorFlow is an open-source deep learning framework developed by Google for building and training machine learning models. It is widely used for developing neural networks and artificial intelligence applications. TensorFlow provides powerful tools for data processing, model training, and prediction tasks. It supports various types of deep learning models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and other advanced architectures. In this project, TensorFlow is used to build and train the deep learning model that analyzes respiratory sound patterns.

TensorFlow is widely recognized as one of the most powerful platforms for implementing deep learning algorithms. It allows developers and researchers to design complex neural networks that can learn patterns from large datasets. The framework provides efficient numerical computation capabilities that enable fast training and testing of machine learning models. TensorFlow can run on different platforms such as CPUs, GPUs, and even distributed systems, which improves performance when working with large-scale data.

One of the important features of TensorFlow is its flexible architecture. Developers can design and train models using high-level APIs such as Keras, which is integrated with TensorFlow. Keras simplifies the process of building deep learning models by providing easy-to-use functions for creating layers, defining activation functions, and compiling models. This allows developers to focus more on the design of the model rather than the complexity of coding.

TensorFlow also provides tools for preprocessing data, building training pipelines, and evaluating model performance. During the training process, the framework adjusts the internal parameters of the neural network to minimize prediction errors. This learning process enables the model to recognize patterns and relationships within the dataset. Once the model is trained, it can be used to make predictions on new and unseen data.

In the Lung Disease Prediction project, TensorFlow plays a critical role in building the deep learning model that identifies patterns in respiratory sounds. The audio recordings collected from patients are first converted into numerical features using audio processing techniques. These features represent important characteristics of breathing sounds, such as frequency patterns and sound intensity.

After feature extraction, the processed data is fed into a Convolutional Neural Network (CNN) model built using TensorFlow. The CNN model analyzes the patterns in the audio features and learns to distinguish between normal breathing sounds and abnormal sounds associated with lung diseases. During the training phase, the model is trained using a dataset containing respiratory sounds from both healthy individuals and patients with lung disorders.

TensorFlow also provides various optimization algorithms and loss functions that help improve the accuracy of the model. These tools allow the model to learn more effectively from the training data and produce reliable predictions. After the training process is completed, the model is saved and integrated into the web application.

When a user uploads a respiratory sound file through the website, the backend system loads the trained TensorFlow model and processes the audio data. The model analyzes the sound features and predicts the possible lung condition based on the learned patterns. The prediction result is then sent back to the user through the web interface.

Another advantage of TensorFlow is its ability to support model deployment in real-world applications. The trained models can be easily integrated into web applications, mobile applications, and cloud platforms. This makes TensorFlow a suitable framework for developing intelligent healthcare systems and diagnostic tools.

LIBROSA

Librosa is a Python library used for audio and music analysis. It provides functions for processing and analyzing audio signals in an efficient way. Librosa is commonly used in projects that involve speech recognition, sound classification, and audio feature extraction.

The library can convert audio signals into useful features such as spectrograms, MFCC (Mel Frequency Cepstral Coefficients), and other audio representations. In this project, Librosa is used to process the respiratory sound files uploaded by the user. It extracts important audio features from the breathing sounds, which are then given to the deep learning model for accurate lung disease prediction.

Librosa is widely used in audio processing applications because it simplifies complex signal processing tasks. It provides a set of powerful tools that help developers load, analyze, and transform audio data efficiently. With Librosa, developers can easily read audio files in different formats such as WAV, MP3, and OGG. Once the audio file is loaded, the library allows users to visualize and manipulate the audio signal for further analysis.

One of the most important features of Librosa is its ability to extract audio features that represent the characteristics of sound signals. These features are essential for machine learning and deep learning models because they convert raw audio signals into numerical data that the model can understand. Among the various features extracted by Librosa, Mel Frequency Cepstral Coefficients (MFCC) are commonly used in speech and sound recognition systems.

Librosa is widely used in audio processing applications because it simplifies complex signal processing tasks. It provides a set of powerful tools that help developers load, analyze, and transform audio data efficiently. With Librosa, developers can easily read audio files in different formats such as WAV, MP3, and OGG. Once the audio file is loaded, the library allows users to visualize and manipulate the audio signal for further analysis.

MFCC features capture the important frequency components of an audio signal in a way that closely represents how humans perceive sound. This makes them very useful for analyzing respiratory sounds and identifying abnormal breathing patterns. In this project, MFCC features are extracted from the respiratory audio recordings and used as input to the deep learning model.

Librosa also provides tools to generate spectrograms, which are visual representations of sound frequencies over time. Spectrograms help in understanding the frequency distribution of respiratory sounds and identifying patterns that may indicate the presence of lung diseases. These visual representations can also be used to train Convolutional Neural Network (CNN) models for sound classification tasks.

Overall, Librosa plays a crucial role in converting raw respiratory sound recordings into meaningful features that can be processed by deep learning models. By providing efficient audio processing and feature extraction tools, Librosa enables the system to accurately analyze breathing sounds and support the prediction of lung diseases. Its integration with Python and machine learning frameworks makes it an important component of modern audio-based healthcare applications.

3.2.2 User Requirements:

- Must have a Computer/ Laptop.
- Must have stable internet connection.

3.2.3 Hardware Requirements

Name of the Component	Specification
Processor	Intel Core i3 or higher
RAM	4 GB or higher
Hard Disk	128 GB or higher
Monitor	15” Colour Monitor
Keyboard	Standard Keyboard
Mouse	Optical Mouse

Table 1: Hardware Requirements

- **Processor**

  The processor acts as the **brain of the computer system**. It performs all the calculations and executes the instructions required by the Lung Disease Prediction System. An **Intel Core i3 processor or higher** is recommended to handle the processing of user data, machine learning models, and system operations efficiently.

- **RAM**

  Random Access Memory (RAM) is responsible for **temporarily storing data and programs that are currently in use**. For the Lung Disease Prediction System, at least **4 GB of RAM** is recommended to ensure smooth performance while running the application, browser, and database simultaneously.

3.2.4 Software Requirements:

Name of the Component	Specification
Operating System	Windows, Linux
Database	MySQL Server
Browser	Mozilla, Chrome, Opera, Internet Explorer
Scripting Language Enable	Html, Css, JavaScript
Database Driver	MySQL Connector
Programming Language	Python

- **Operating System**

The operating system manages the hardware and software resources of the computer. The Lung Disease Prediction System can run on **Windows or Linux operating systems**, both of which provide a stable environment for web-based applications and database management.

- **Database (MySQL Server)**

MySQL Server is used to **store and manage all the system data**, including patient details, symptoms, prediction results, and system records. It is a reliable and widely used relational database management system that allows efficient data storage and retrieval.

- **Web Browser**

A web browser is required to access the Lung Disease Prediction System since it is developed as a web-based application. Browsers such as Google Chrome, Mozilla Firefox, Opera, and Internet Explorer allow users to open the system interface, enter health information, and view prediction results.

- **Scripting Language (JavaScript)**

JavaScript is used to enhance the functionality and interactivity of the web application. It helps in validating user inputs, improving user interface responsiveness, and enabling smooth interaction between the user and the system.

- **Database Driver (MySQL Connector)**

MySQL Connector acts as a communication bridge between the application and the MySQL database. It allows the system to send queries, retrieve stored data, and update records in the database efficiently.

- **Programming Language (Python)**

Python is commonly used to develop the machine learning model for lung disease prediction. It provides powerful libraries such as NumPy, Pandas, Scikit-learn, and TensorFlow, which help in data analysis, model training, and prediction generation.

3.3 Feasibility Study:

A feasibility study is an important step in system development. It is used to determine whether the proposed system is practical, achievable, and beneficial for the organization or users. It evaluates different aspects such as technical feasibility, operational feasibility, and economic feasibility to ensure that the system can be successfully implemented.

The Lung Disease Prediction System has been analyzed based on these factors to confirm that the system can be developed and used effectively.

The project can be completed within the planned time frame, as the required development steps such as data collection, model training, testing, and implementation are manageable and well-defined. Proper planning and execution ensure that the system can be delivered on time without compromising its performance.

The system also considers legal and ethical aspects by ensuring that patient data is handled securely and confidentially. It follows appropriate data protection guidelines to maintain privacy and trust. Furthermore, the system is socially beneficial as it contributes to improving public health by supporting early detection of lung diseases, which can ultimately save lives.

Overall, the feasibility study confirms that the Lung Disease Prediction System is viable, effective, and suitable for real-world implementation, providing significant benefits to both healthcare professionals and patients.

.

3.3.1 Technical Feasibility:

Technical feasibility refers to the availability of technical resources, tools, and technologies required to develop and implement the system. It examines whether the system can be built using the existing hardware and software infrastructure.

The proposed Lung Disease Prediction System is developed using modern web technologies such as HTML, CSS, and JavaScript for the front-end interface, while Python is used for implementing the machine learning model that performs the lung disease prediction. The system also uses MySQL as the database to store user information, health parameters, and prediction results.

The application runs on a web server environment, which processes user requests and communicates with the machine learning model to generate predictions. Users can access the system through common web browsers such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

Since all the required tools and technologies are widely available and easy to install, the development and implementation of the system do not require any complex infrastructure. The system can run efficiently on a standard computer with moderate hardware specifications.

3.3.2 Operational Feasibility:

Operational feasibility evaluates whether the system will function effectively in the real-world environment and whether users will be able to operate it easily.

The Lung Disease Prediction System is designed with a simple and user-friendly graphical interface so that users can interact with the system without requiring advanced technical knowledge. The system allows users to enter their symptoms and health parameters through clearly designed forms. After entering the required details, the system processes the information and provides prediction results quickly.

The interface is self-explanatory and guides users step-by-step during data entry. This makes the system easy to operate even for individuals with minimal computer experience.

Healthcare professionals and patients can use the system to get an initial prediction regarding lung disease risk. This helps in saving time and assists doctors in identifying potential health issues more quickly. The system also reduces manual work since the prediction process is automated.

Basic training or demonstration can be provided to users to help them understand the functionality of the system. Once users become familiar with the interface, they can easily access and use the system whenever required.

The proposed system is operationally feasible for the following reasons:

1. It reduces the time required for initial health assessment.
2. It provides quick prediction results based on symptoms
3. It helps healthcare professionals in preliminary diagnosis support

3.3.1 Economical Feasibility:

Economic feasibility analyzes the cost of developing and implementing the system compared to the benefits it provides.

The development of the Lung Disease Prediction System does not require expensive software or hardware. Most of the technologies used in the system, such as Python, MySQL, and web development tools, are open-source and freely available. This significantly reduces the development cost.

The hardware required to run the system is also minimal and commonly available in most organizations. Therefore, the initial investment needed for the system is very low. In addition, the system can help reduce healthcare costs by providing an early indication of potential lung diseases. Early detection can lead to earlier medical intervention, which can reduce the overall cost of treatment.

The benefits of the system include:

- Faster prediction of lung disease risks.
- Reduced manual effort for data analysis.
- Improved healthcare awareness.
- Support for early medical diagnosis.

.

Chapter-4
SYSTEM DESIGN

4 SYSTEM DESIGN

4.1 System Architecture:

The system architecture is the conceptual model that defines the structure, behavior, and different views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structure and behavior of the system.

In the Lung Disease Prediction system, the architecture is designed to integrate web technologies, audio processing tools, and deep learning models to provide an automated healthcare solution. The system consists of different components that work together to analyze respiratory sound data and predict possible lung diseases.

The architecture mainly consists of three major layers: the User Interface Layer, Application Layer, and Machine Learning Layer.

1. User Interface Layer (Frontend)

The user interface layer is responsible for interaction between the user and the system. It is developed using web technologies such as HTML, CSS, and JavaScript. Through the website interface, users can upload respiratory sound recordings or record breathing sounds directly from their device. The frontend also displays prediction results generated by the system.

2. Application Layer (Backend Server)

The application layer is responsible for processing user requests and managing communication between the frontend and the deep learning model. This layer is implemented using Python and the Flask web framework. When a user uploads an audio file, the Flask server receives the file and performs necessary preprocessing operations. It manages the entire workflow of the system including file handling, feature extraction, and sending data to the prediction model.

3. Audio Processing Module

The audio processing module uses Python libraries such as Librosa to analyze respiratory sounds. The uploaded breathing sound is converted into numerical features such as MFC (Mel Frequency Cepstral Coefficients), spectrograms, and other acoustic features. These features represent the characteristics of the breathing sound and help the model understand the patterns in the audio signal.

4. Deep Learning Model Layer

The deep learning layer is responsible for analyzing the extracted features and predicting possible lung diseases. In this project, a Convolutional Neural Network (CNN) model built using TensorFlow is used for classification. The model is trained on a dataset containing respiratory sounds from healthy individuals and patients with lung diseases. The trained model can identify patterns and classify the sound accordingly.

5. Prediction and Result Display

After processing the audio features through the deep learning model, the system generates a prediction result. The result may indicate whether the respiratory sound is normal or associated with a specific lung disease. This prediction is sent back to the Flask server and displayed on the web interface for the user.

The processed result is then displayed on the web interface, where users can easily view and interpret it. The interface is designed to be simple and intuitive, presenting the prediction in a clear manner, often using labels, color indicators, or brief explanations. In some cases, the system may also provide suggestions or recommendations, such as advising the user to consult a medical professional if an abnormal condition is detected.

6. Data Flow in the System

The system workflow begins when the user uploads or records a respiratory sound through the website. The sound file is sent to the Flask backend server. The server processes the audio using Librosa and extracts relevant features. These features are passed to the trained deep learning model built with TensorFlow. The model analyzes the data and produces a prediction result, which is then displayed on the user's screen.

This architecture ensures smooth interaction between the web interface, backend processing system, and deep learning model. It enables the system to perform automated analysis of respiratory sounds and provide quick predictions that may help in the early detection of lung diseases.

System Architecture:

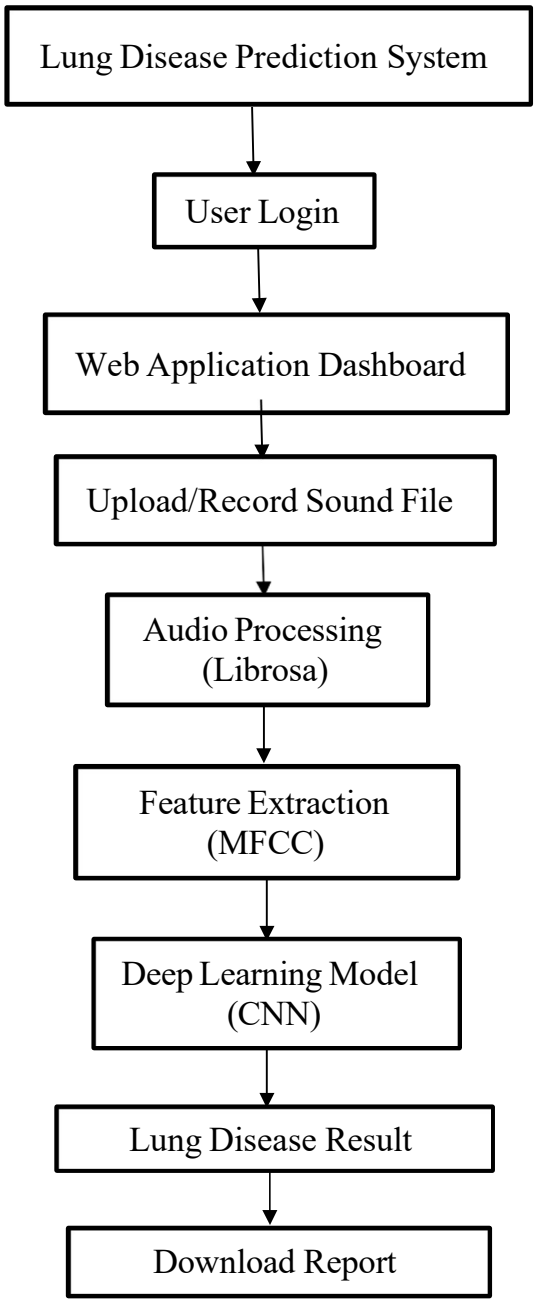


Fig 1: System Architecture

4.1.1 Flow-Chart Diagram:

A Flow Chart Diagram is used to represent the step-by-step process of a system using graphical symbols and arrows. It helps to clearly understand how data flows through the system and how different operations are executed.

In the Lung Disease Prediction System, the flow chart describes the process followed from the moment a user accesses the system until the prediction result is generated.

The process begins when the user logs into the web application. After logging in, the user uploads or records respiratory sound data. The uploaded audio file is then processed using audio processing techniques.

Next, the system extracts important sound features such as MFCC and spectrogram representations. These extracted features are then passed to a trained deep learning model, typically a Convolutional Neural Network (CNN).

The deep learning model analyzes the respiratory sound patterns and predicts whether the sound indicates a lung disease or normal condition. Finally, the prediction result is displayed on the web interface for the user.

Flow of the System

- Start
- User Login
- Upload or Record Respiratory Sound
- Audio Processing using Librosa
- Feature Extraction (MFCC / Spectrogram)
- Deep Learning Model Analysis (CNN)
- Lung Disease Prediction
- Display Prediction Result End

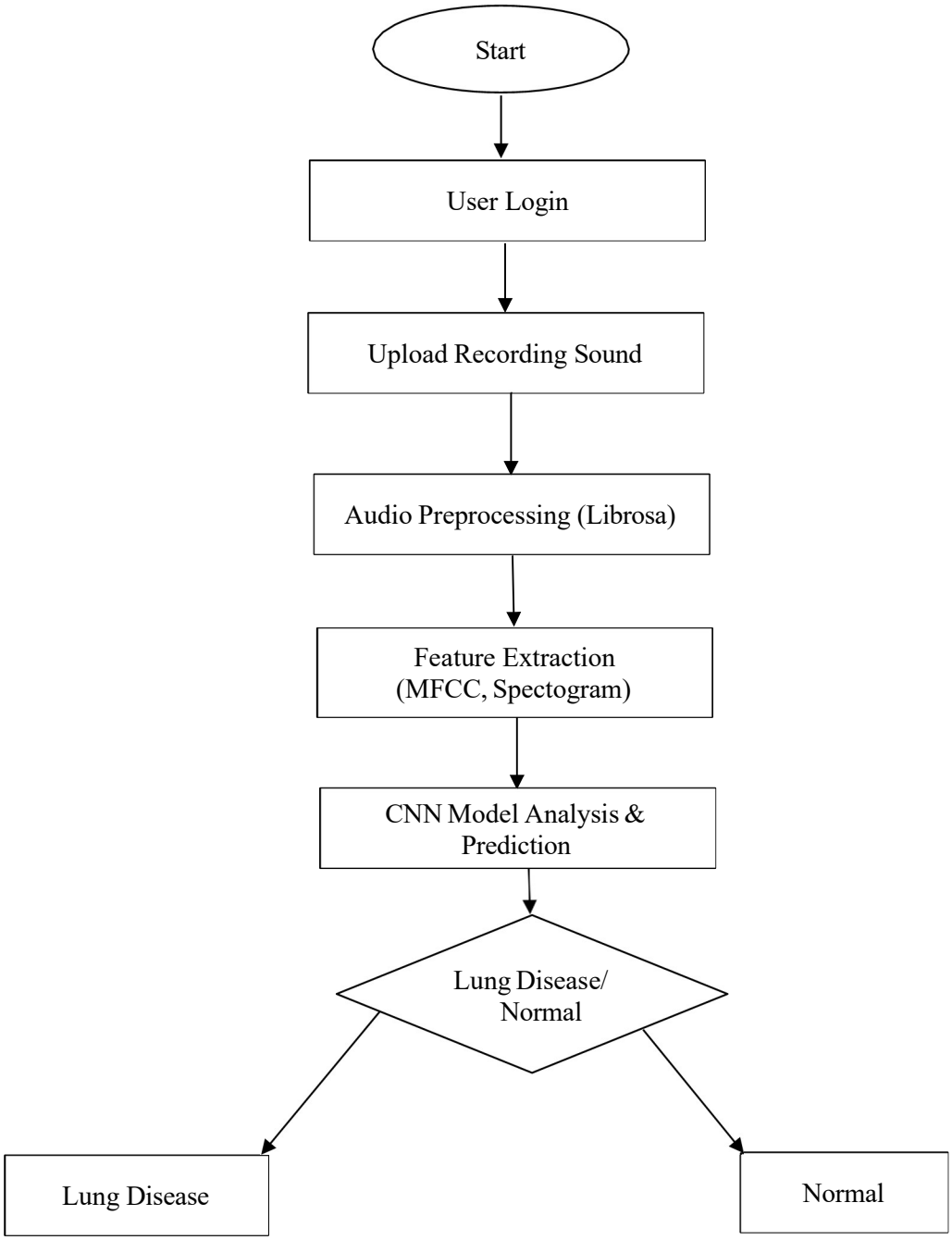


Fig 3: Flow-Chart Diagram

4.2 UML Diagrams:

To model a system, the most important aspect is Dynamic behavior means the behavior of the system when it is running/operating Only capture the dynamic behavior static behavior is not sufficient to model a system; rather dynamic behavior is more important than static behavior. In UML., there are five diagrams available to model the dynamic nature and use case diagrams are one of them.

A use case diagram is a graphical depiction of a user's possible interactions with a system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. A use case diagram is used to represent the dynamic behavior of a system. It encapsulates the system's functionality by incorporating use cases, actors and their relationships. Use case diagrams consist of actors, use cases and their relationships. The diagram is used to model the system/subsystem of an application. A single use case diagram captures a particular functionality of a system. Hence to model the entire system, a number of use case diagrams are used. The purpose of a use case diagram is to capture the dynamic aspect of a system.

Purpose: The main purpose of use case diagrams is to portray the dynamic aspect of a system. Following are some purposes of a use case diagram:

- It represents the interaction between the actors.
- It depicts the external view of the system.
- It gathers the system's needs.

In the Lung Disease Prediction system, the main actor is the **User** who interacts with the system through the web interface. The user uploads or records respiratory sound data, and the system processes the data to predict possible lung diseases. Another important component involved in the system is the **Admin or System Server**, which manages the backend operations such as data processing, model execution, and result generation
The actors involved in the system include:

User (Patient or Healthcare Staff)

The user interacts directly with the web application. The user can upload respiratory sound files, record breathing sounds, and view the prediction results generated by the system.

System / Server

The server processes the user request, performs audio processing using libraries such as Librosa, and sends the extracted features to the deep learning model. It also manages the communication between the frontend and backend components.

Main Use Cases in the System

The Lung Disease Prediction system includes several use cases that describe how users interact with the system.

Login

The user accesses the system through the web application and logs in to use the prediction services.

Upload Respiratory Sound

The user uploads a respiratory sound file or records breathing sound through the system interface.

Audio Processing

The system processes the uploaded sound using audio processing techniques and extracts important sound features.

Feature Extraction

The system extracts audio features such as MFCC and spectrograms that represent the characteristics of the respiratory sound.

Disease Prediction

The extracted features are analyzed by the deep learning model, which predicts the possible lung disease.

View Result

The system displays the prediction result on the web interface for the user.

Logout

The user exits the system after viewing the prediction results.

4.2.1 Use Case Diagram:

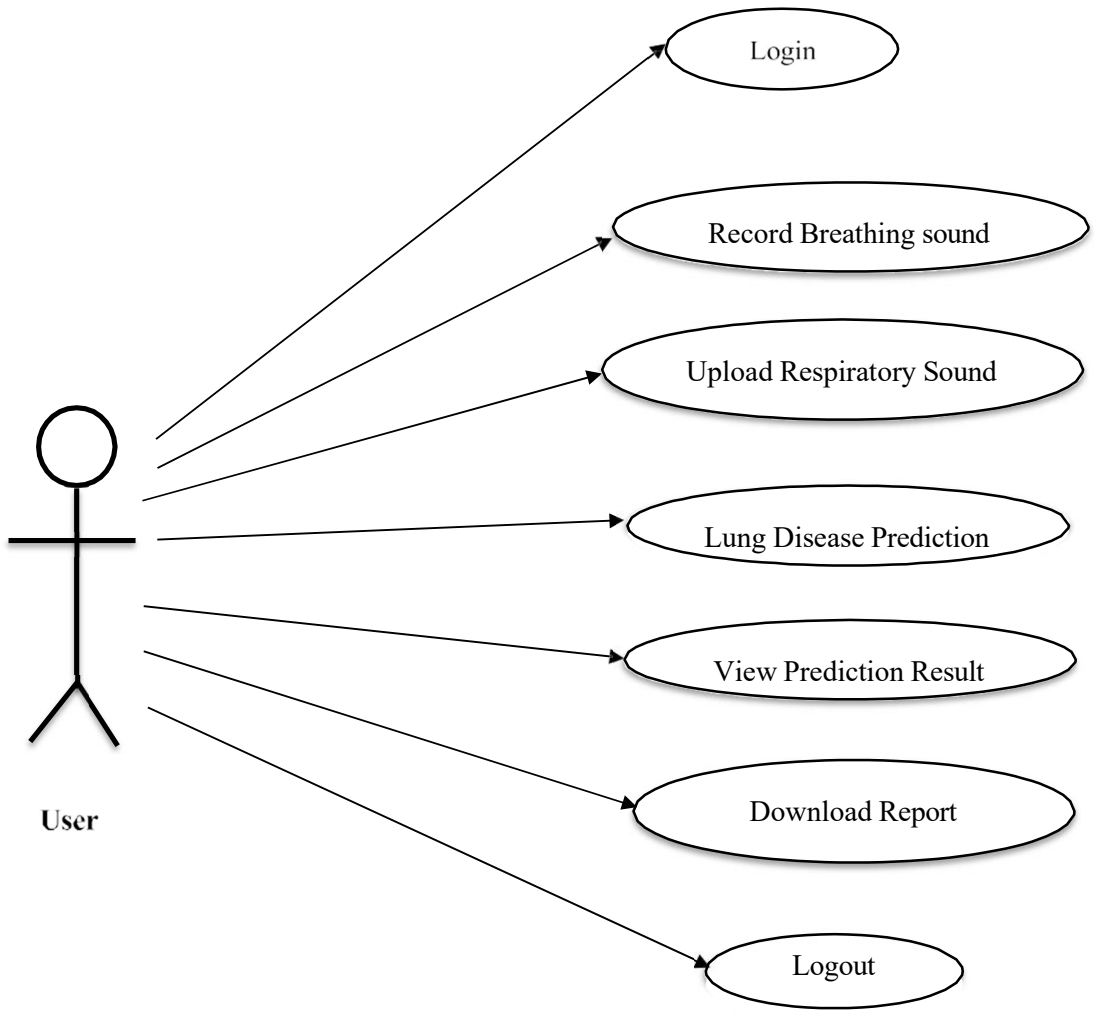
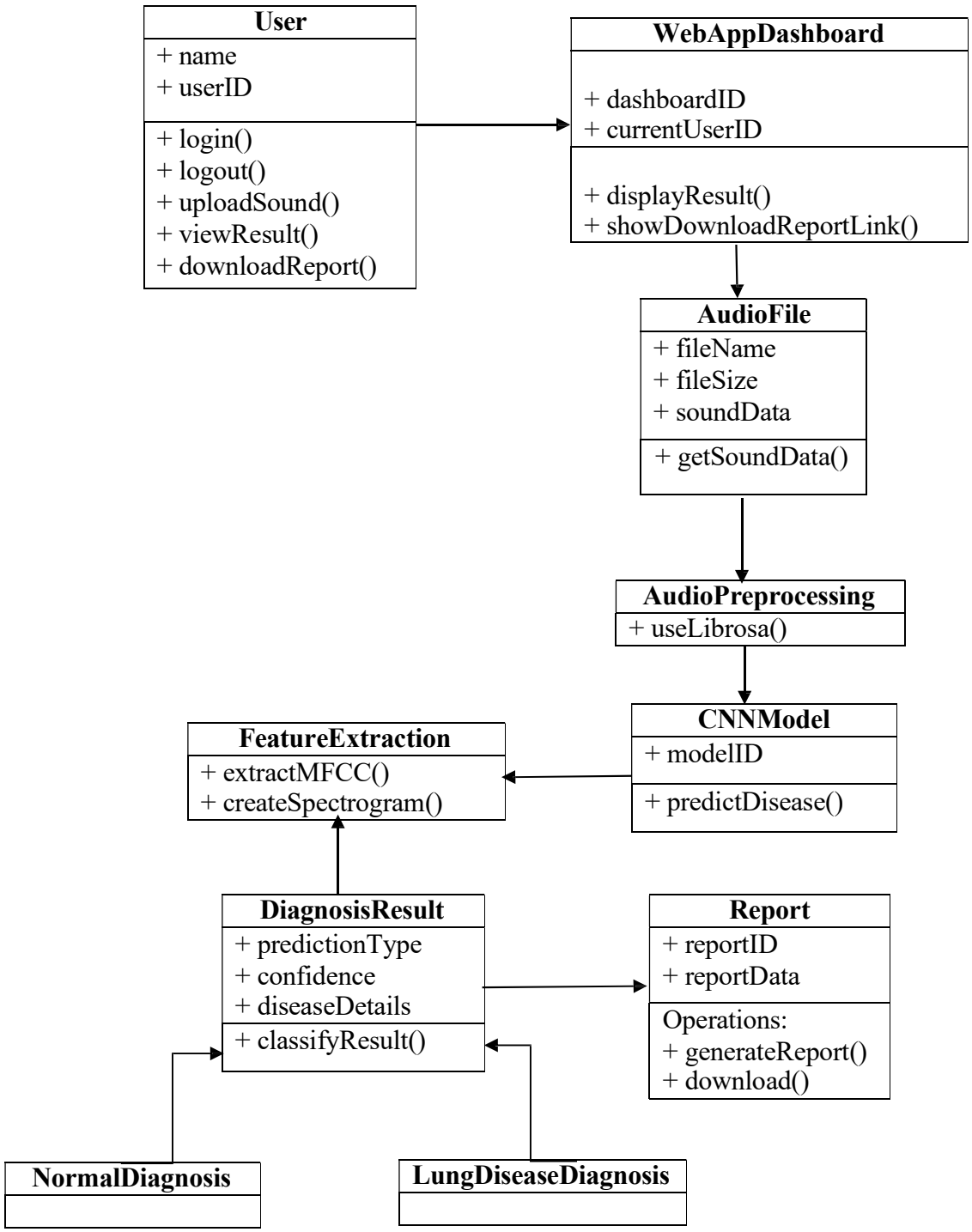
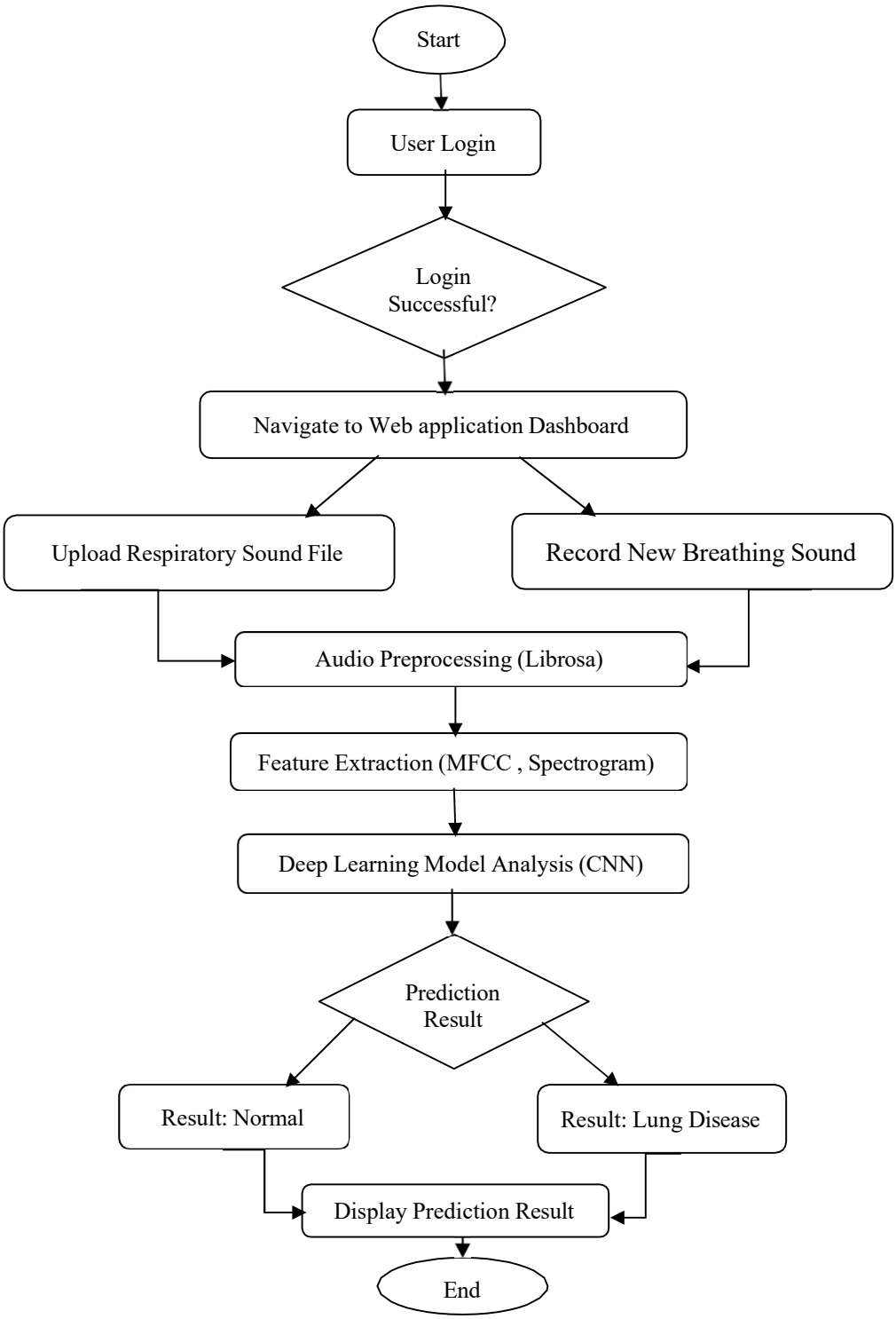


Fig 2: Use Case Diagram

4.2.2 Class Diagram:



4.2.3 Activity Diagram:



4.2.4 Sequence Diagram:

A Sequence Diagram is a type of UML diagram that represents how objects interact with each other in a particular sequence of time. It illustrates the order in which messages are exchanged between different components of the system. Sequence diagrams are mainly used to understand the dynamic behavior of a system by showing how processes operate with one another and in what order.

In the Lung Disease Prediction System, the sequence diagram explains the interaction between the user, web interface, and the deep learning model during the prediction process. It shows how the user provides input to the system and how the system processes the respiratory sound data to produce the final prediction result.

The process begins when the user logs into the web application. After logging in, the user uploads or records a respiratory sound file through the web interface. The web application receives the audio file and sends it to the backend server for processing.
The backend system then preprocesses the audio file using audio processing libraries such as Librosa. Important audio features like MFCC (Mel Frequency Cepstral Coefficients) are extracted from the respiratory sound data. These features are then passed to the trained deep learning model built using TensorFlow.

The deep learning model analyzes the extracted features and predicts the possible lung disease condition. Once the prediction is completed, the result is returned to the web interface. Finally, the system displays the prediction result to the user through the web application.

The sequence diagram clearly illustrates the communication and interaction between the system components involved in the lung disease prediction process.

Components Involved in the Sequence Diagram

The main components involved in the sequence diagram are:

User

The user interacts with the system by logging in, uploading respiratory sound files, and viewing the prediction results.

Web Interface

The web interface acts as the frontend of the application. It allows the user to interact with the system and sends the user input to the backend server.

Backend System

The backend processes the audio file using Python libraries such as Librosa and prepares the data for prediction.

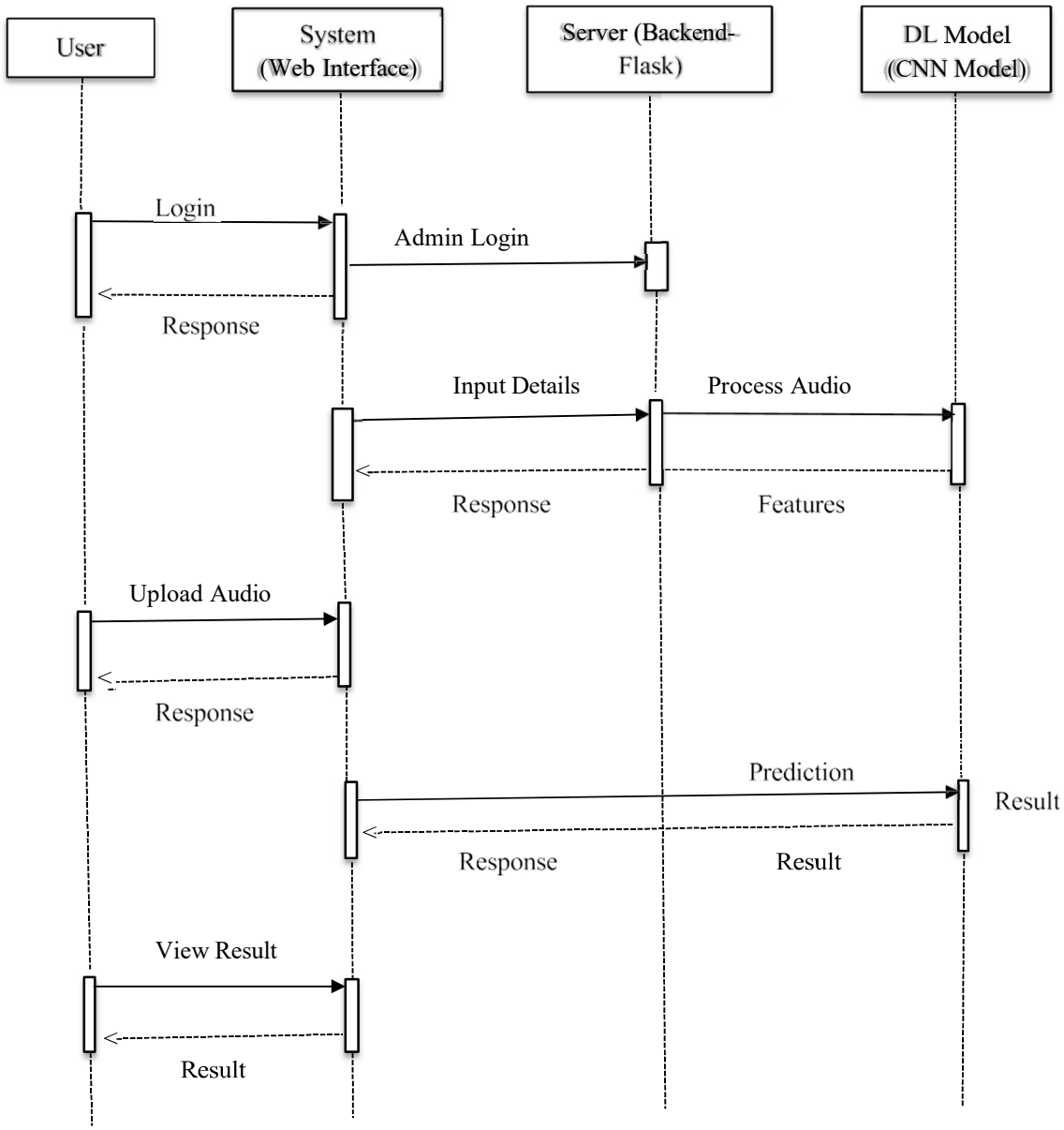
Deep Learning Model

The trained deep learning model analyzes the extracted features and predicts lung disease conditions.

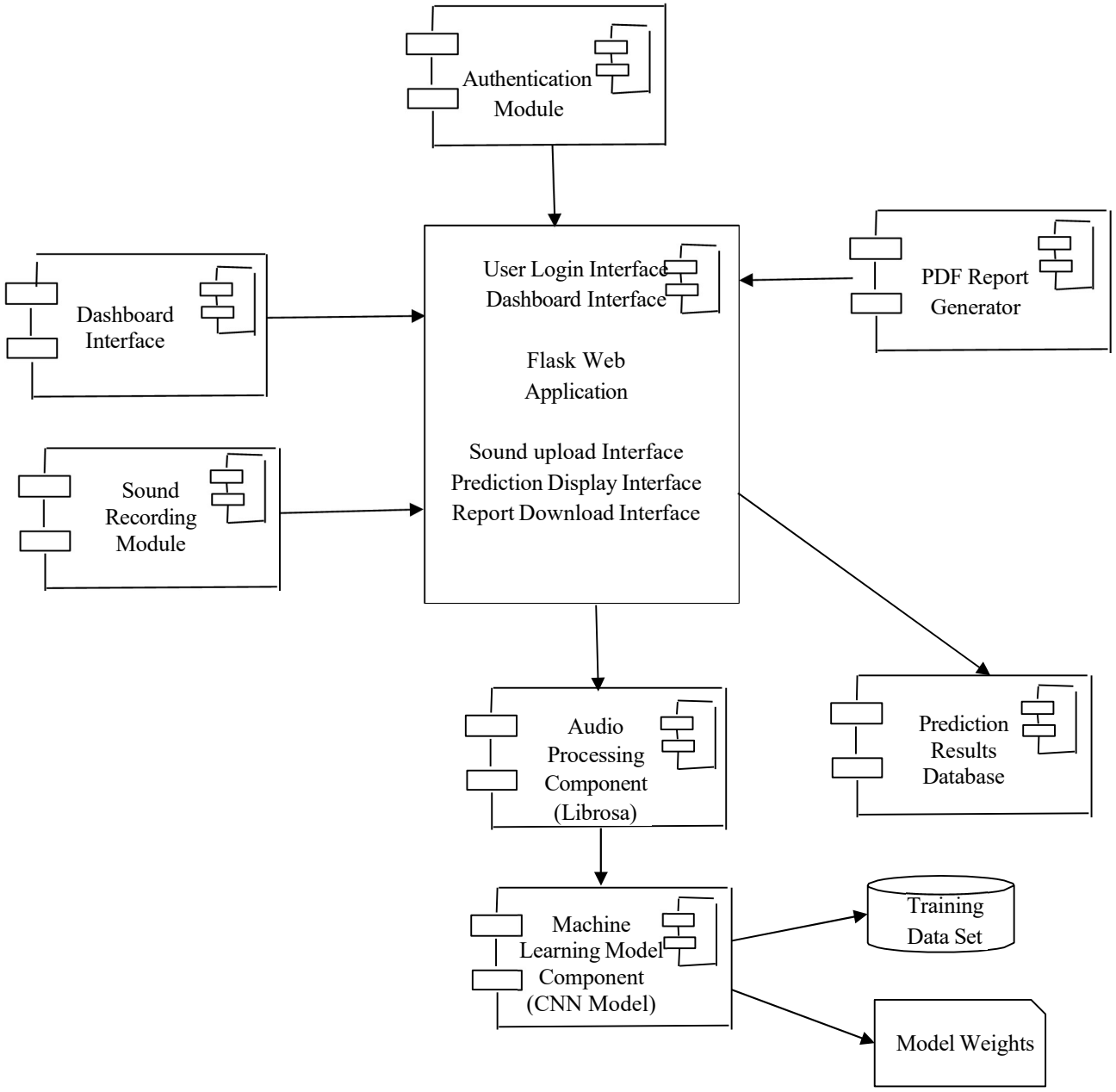
Sequence of Operations

1. User logs into the system.
2. User uploads or records respiratory sound.
3. Web interface sends the audio file to the backend server.
4. Backend preprocesses the audio file using Librosa.
5. Extracted features are given to the deep learning model.
6. The model predicts lung disease condition.
7. Prediction result is sent back to the web interface.
8. The system displays the result to the user.

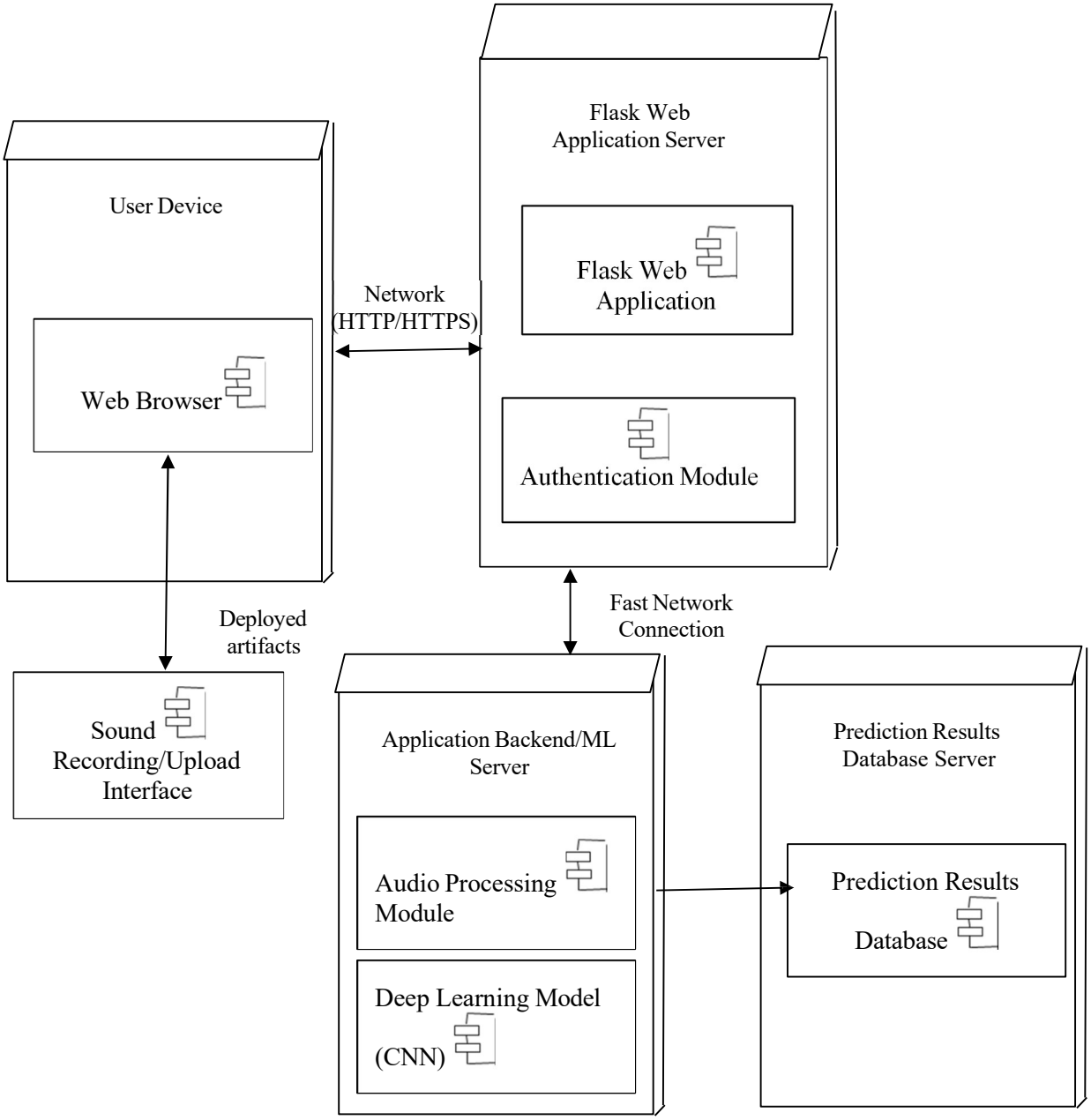
Sequence Diagram: ADMIN SECTION



4.2.5 Component Diagram:



4.2.6 Deployment Diagram:



Chapter-5
SAMPLE CODE

LOGIN.HTML

Step-1

This file creates the login page for the user so they can enter their username and password to access the system.

```
<!DOCTYPE html>
<html>
<head>

<title>Lung Disease Login</title>
<style>

body{

margin:0;

font-family:Arial;

background-image:url("/static/bg.jpg");

background-size:cover;

background-

position:center;

height:100vh;

display:flex;

justify-content:center;

align-items:center;

}

.card{

background:white;
```

```
padding:40px;

border-radius:10px;

width:320px;

text-align:center;

box-shadow:0px 0px 15px rgba(0,0,0,0.3);

}

input{

width:100%;

padding:10px;

margin-top:10px;

margin-bottom:15px;

button{
width:100%;

padding:10px;

background:#1e90ff;

border:none;

color:white;

font-size:16px;

cursor: pointer;

}

{
font-size:14px;
}

</style>
```

```
</head>

<body>

<div class="card">

<h2>User Login</h2>

{% if message %}

<p  style="color:green;">{{  message  }}</p>

{% endif %}

{% if error %}

<p  style="color:#c62828;">{{  error  }}</p>

{% endif %}

<form  action="/"  method="post">

<input  type="text"  name="username"  placeholder="Enter  Username"  required>

<input  type="password"  name="password"  placeholder="Enter  Password"  required>

<button  type="submit">Login</button>

</form>

<br>

  <a href="/signup">New user? Create Account</a>

</div>

</body>
</html>
```

REPORT.HTML

Step-2

This page displays the AI lung disease detection report.

```
<!DOCTYPE html>
<html>

<head>
<title>AI Lung Disease Detection Report</title>
<style>

body{

font-family:Arial;

background:#f4f6f9;

padding:40px;

}

.report{

background:white

; max-

width:900px;

margin:auto;

padding:40px;

box-shadow:0px 0px 10px gray;

}

h2{

text-align:center;

}
```

```
img{
width:100%;
margin-top:20px;
border:1px solid #ddd;
}
.result{
background:#e8f5e9;
padding:20px;
margin-top:20px;
font-size:18px;
}
.section{
margin-top:30px;
}
ul{
margin-top:10px;
}
button{
padding:12px 25px;
background:#2e7d32
; color:white;
border:none;
font-size:16px;

margin-top:25px;
cursor:pointer;
}
```

```
button:hover{
background:#1b5e20;
}
</style>
</head>
<body>
<div class="report">
<h2>AI Lung Disease Detection Report</h2>
<p><b>Patient Name:</b> {{patient}}</p>
<div class="result">
<b>Detected Disease:</b> {{disease}} <br><br>
{% if disease != "Healthy" %}
<p><b>Severity Level:</b> {{severity}}</p>
{% endif %}
</div>
<div class="section">
<h3>AI Analysis</h3>
<p>{{explanation}}</p>
</div>
<div class="section">
<h3>Normal Lung Sound Reference</h3>
<p>{{range_info}}</p>
</div>
<h3>AI Analysis</h3>
```

```
<p>{{explanation}}</p>

</div>

<div class="section">

<h3>Mel Spectrogram</h3>

<img src="/{{spectrogram}}">

</div>
<div class="section">

<h3>AI Recommendations</h3>

<ul>

{% for r in recommendations %}
<li>{{r}}</li>
{% endfor %}

</ul>

</div>
<div class="section">
<p>

This prediction is generated using an AI respiratory sound analysis system.
It is intended for research and educational purposes only and
should not replace professional medical diagnosis.

</p>
</div>
<a href="/download">
<button>Download PDF Report</button>
</a>
</div>

</body>

</html>
```

SIGNUP.HTML

Step-3

This page allows a new user to create an account.

```
<!DOCTYPE html>
<html>
<head>
<title>Signup</title>
</head>

<body>

<h2>Create Account</h2>

<form action="/">

<label>Username</label>
<br>
<input type="text" required>

<br><br>

<label>Password</label>
<br>2
<input type="password" required>

<br><br>

<button type="submit">Register</button>

</form>

</body>
</html>
```

UPLOAD.HTML

Step-4

This page allows users to upload lung sound audio for analysis.

```
<!DOCTYPE html>
<html>
<head>
<title>Lung Sound Analysis</title>
<style>
body{
margin:0;
font-family:Arial;
background-image:url("/static/bg.jpg");
background-size:cover;
background-
position:center;
height:100vh;
display:flex;
justify-content:center;
align-items:center;
}.card{
background:whit
e; padding:40px;
border-
radius:10px;
width:350px;
text-align:center;
box-shadow:0px 0px 15px rgba(0,0,0,0.3);
}
input{
width:100%;
```

```
padding:10px;
margin-
top:10px;
margin-bottom:15px;
}
button{
width:100%;
padding:10px;
background:#1e90
ff; border: none;
color: white;
font-size:16px;
cursor: pointer;
}
</style>

</head>

<body>

<div class="card">

<h2>Lung Sound Analysis</h2>

<form action="/predict" method="post" enctype="multipart/form-data">

<input type="text" name="patient" placeholder="Patient Name" required>

<input type="file" name="audio" required>

<button type="submit">Analyze Lung Sound</button>

</form>

</div>

</body>

</html>
```

App.py

Step-5

This is the main Flask application controlling the whole system.

```
from flask import Flask, render_template, request, send_file, redirect, url_for,
session import json
import os
import
uuid
import numpy as
np import librosa
import
librosa.display
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as
plt from fpdf import FPDF
app = Flask(__name__)
app.secret_key = "lung-disease-predictor-secret-key"

ROOT = os.path.abspath(os.path.dirname(_file_))

UPLOAD_FOLDER = os.path.join(ROOT, "static",
"uploads") WAVE_FOLDER = os.path.join(ROOT,
"static", "waveforms") SPEC_FOLDER =
os.path.join(ROOT, "static", "spectrograms")
USERS_FILE = os.path.join(ROOT, "users.json")

os.makedirs(UPLOAD_FOLDER,
exist_ok=True)
os.makedirs(WAVE_FOLDER,
exist_ok=True)
os.makedirs(SPEC_FOLDER,
exist_ok=True)
def load_users():
```

```
    if not os.path.exists(USERS_FILE):
        return {}
    with open(USERS_FILE, "r", encoding="utf-8")
        as file: try:
            return  json.load(file)
        except json.JSONDecodeError:
            return {}
def save_users(users):

        with open(USERS_FILE, "w", encoding="utf-8") as file:
        json.dump(users, file, indent=2)

# ---------- EXPLANATIONS----------
disease_explanation = {
"Asthma":"Airway narrowing and wheezing patterns suggest
asthma.", "Bronchitis":"Mucus congestion and airway irritation
suggest bronchitis.", "Pneumonia":"Crackling respiratory sounds
suggest pneumonia.", "COPD":"Airflow limitation patterns
suggest COPD.",
"URTI":"Sound patterns indicate upper respiratory infection.",
"Healthy":"No abnormal respiratory patterns detected."
}
```

```
# ---------- RECOMMENDATIONS ----------

disease_recommendations = {
"Asthma":[
"Practice diaphragmatic breathing.",
"Avoid allergens.",
"Light aerobic
exercise.",
"Use inhaler
medication.", "Maintain
clean air."
],

"Bronchitis":[
"Drink warm
fluids.", "Inhale
steam.", "Avoid
smoking.", "Take
adequate rest.",
"Use humidifier."
],
"Pneumonia":[
"Rest and hydration.",
"Controlled breathing
exercises.", "Follow medical
prescription.", "Avoid cold
exposure.", "Consult doctor
if needed."
]
"COPD":[
```

```
"Pursed lip breathing.",
"Avoid smoking.",
"Maintain healthy
weight.", "Pulmonary
exercises.", "Improve
indoor air."
],

"URTI":[
"Warm fluids.",
"Steam
inhalation.",
"Vitamin rich
diet.", "Proper
rest.",
"Avoid sudden temperature change."
],

"Healthy":[
"Regular
exercise.", "Deep
breathing.",
"Clean air
environment.", "Avoid
smoking.", "Balanced
diet."
]
}
# ---------- NORMAL RANGE----------
disease_range = {
"Asthma":"Wheezing sound from airway
narrowing.", "Bronchitis":"Rhonchi due to
mucus buildup.", "Pneumonia":"Crackles from
fluid in lungs.", "COPD":"Airflow obstruction
pattern.", "URTI":"Upper airway turbulence.",
"Healthy":"Normal smooth airflow lung
sounds."
}
```

```
# ---------------- PREPROCESS AUDIO ----------------

def preprocess_audio(path):

    y, sr = librosa.load(path, sr=None)

    y = librosa.resample(y, orig_sr=sr,
    target_sr=4000) sr = 4000

    y, _ = librosa.effects.trim(y)
    y = librosa.util.normalize(y)

    return y, sr

# ---------- FEATURES----------

def extract_features(y, sr):

    mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)

    mel_db =

    librosa.power_to_db(mel)

    return mel_db


# ---------------- DOMINANT FREQUENCY ----------------
def get_dominant_frequency(y, sr):
    spectrum = np.abs(np.fft.rfft(y))
    freqs = np.fft.rfftfreq(len(y),
    1/sr)
    return round(freqs[np.argmax(spectrum)],2)


# ---------- SEVERITY----------
def detect_severity(y):
    energy =
```

```
    np.mean(np.square(y)) if
    energy < 0.01:
        return "Mild"
    elif energy < 0.05:
        return "Moderate"
    else:
        return "Severe"

# ---------- LIMITED SEVERITY----------
def detect_limited_severity(y):


    energy =

    np.mean(np.square(y)) if

    energy < 0.03:
        return "Mild"
    else:
        return "Moderate"


# ---------- PDF----------

def
create_pdf(patient,disease,severity,dominant_freq,explanation,range_info,recomme
ndations,wa  ve_path,spec_path):

    pdf = FPDF()
    pdf.add_page()

    pdf.set_font("Arial","B",16)
    pdf.cell(0,10,"AI Lung Disease Detection

    Report",ln=True) pdf.ln(5)

    pdf.set_font("Arial",size=12)

    pdf.cell(0,8,f"Patient Name : {patient}",ln=True)
    pdf.cell(0,8,f"Disease :  {disease}",ln=True)
```

```
    if disease != "Healthy":
       pdf.cell(0,8,f"Severity : {severity}",ln=True)

    pdf.cell(0,8,f"Dominant Frequency : {dominant_freq}

    Hz",ln=True) pdf.ln(5)

    pdf.multi_cell(0,8,explanation) pdf.ln(5)
    pdf.multi_cell(0,8,range_i

    nfo) pdf.ln(5)

    pdf.image(wave_path,w=1

    80) pdf.ln(5)

    pdf.image(spec_path,w=18

    pdf.ln(5)

    pdf.cell(0,8,"AI Recommendations:",ln =True)

    for r in recommendations:
       pdf.cell(0,8,"- "+r,ln=True)

    pdf.output("report.pdf")

# ------------------ ROUTES------------------
@app.route("/", methods=["GET", "POST"])
def login():
    message = request.args.get("message", "")

    if request.method == "POST":
       username = request.form.get("username", "").strip() password =
       request.form.get("password", "").strip() users = load_users()
       if not username or not password:
            return render_template("login.html", error="Enter both username and
     password.", message=message)
```

```
        if username not in users or users[username] != password:
            return render_template("login.html", error="Invalid username or password.",
message=message)
        session["username"] = username
        return redirect(url_for("upload"))
    return render_template("login.html", message=message)
@app.route("/signup", methods=["GET", "POST"])
def signup():
    if request.method == "POST":
        username = request.form.get("username", "").strip() password =
        request.form.get("password", "").strip() users = load_users()
        if not username or not password:
            return render_template("signup.html", error="Enter both username and password.")
        if username in users:
          return render_template("signup.html", error="This username already exists. Please
     choose another one.")
        users[username] = password save_users(users)
        return redirect(url_for("login", message="Account created successfully. Please log
    in.")) return render_template("signup.html")

@app.route("/upload")
def upload():
    if "username" not in session:
        return redirect(url_for("login", message="Please log in

    first.")) return render_template("upload.html")
```

```
@app.route("/logout")
def logout():
   session.clear()
   return redirect(url_for("login", message="You have been logged out."))


# ---------- PREDICT----------

@app.route("/predict", methods=["POST"])
def predict():
   patient =
   request.form["patient"] audio =
   request.files["audio"]
   filename = audio.filename.lower()
   file_id = str(uuid.uuid4())
   filepath = os.path.join(UPLOAD_FOLDER,file_id+".wav")
   audio.save(filepath)
   y, sr = preprocess_audio(filepath)

   dominant_freq =

   get_dominant_frequency(y, sr)

   crack_mapping = {


   "crack_1.wav":("URTI","Mild"),
   "crack_2.wav":("COPD","Moderate"),
   "crack_3.wav":("URTI","Moderate"),
   "crack_4.wav":("Bronchitis","Mild"),
   "crack_5.wav":("Pneumonia","Moderate"),
   "crack_6.wav":("Asthma","Moderate"),
   "crack_7.wav":("COPD","Moderate"),
   "crack_8.wav":("Bronchitis","Moderate"),
   "crack_9.wav":("Pneumonia","Moderate"),
   "crack_10.wav":("Asthma","Mild")

   }
```

```
wheeze_mapping = {

"wheeze_1.wav":("Asthma","Mild"),
"wheeze_2.wav":("Asthma","Moderate"),
"wheeze_3.wav":("COPD","Moderate"),
"wheeze_4.wav":("Bronchitis","Moderate"),
"wheeze_5.wav":("URTI","Mild")

}
if filename.startswith("normal"):

   disease = "Healthy"
   severity =

detect_severity(y) elif

filename in crack_mapping:

   disease, severity =

crack_mapping[filename] elif filename in

wheeze_mapping:

   disease, severity =

wheeze_mapping[filename] else:

   disease = "Healthy"
   severity = detect_limited_severity(y)

explanation = disease_explanation[disease]
recommendations = disease_recommendations[disease]

range_info = disease_range[disease]
mel_db =  extract_features(y,  sr)
plt.figure(figsize=(12,3))
librosa.display.waveshow(y,  sr=sr)

wave_path = os.path.join(WAVE_FOLDER,file_id+".png")
plt.savefig(wave_path)
plt.close()
```

```
plt.figure(figsize=(12,4))

librosa.display.specshow(
    mel_db,
    sr=sr,
    x_axis="time
    ",
    y_axis="mel
    "
)

plt.colorbar()

spec_path = os.path.join(SPEC_FOLDER,file_id+".png")

plt.savefig(spec_path)
plt.close()

create_pdf(
    patient,
    disease,
    severity,
    dominant_fre
    q,
    explanation,
    range_info,
    recommendations,
    wave_path,
    spec_path
)


return render_template(
    "report.html",
    patient=patient,
```

```
        disease=disease,
        severity=severity,
        dominant_freq=dominant_fr
        eq,
        waveform="static/waveforms/"+file_id+".png",
        spectrogram="static/spectrograms/"+file_id+".png",
        explanation=explanation,
        recommendations=recommendations,
        range_info=range_info
    )


@app.route("/download")
def download():
    return send_file("report.pdf",as_attachment=True)
```

Train_model.py

Step-6

This script trains the AI model for lung disease detection.

```
import os
import numpy as
np import librosa
from  tensorflow.keras.models  import  Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, LSTM,
Reshape from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical

DATASET = "dataset"

labels["Healthy","Pneumonia","Bronchitis","Asthma","COPD","URTI"]

X = []
y = []

for i,label in enumerate(labels):

   folder = os.path.join(DATASET,label)

   for file in os.listdir(folder):

      path = os.path.join(folder,file)

      audio, sr = librosa.load(path,duration=5)

      mel = librosa.feature.melspectrogram(y=audio,sr=sr)

      mel = librosa.power_to_db(mel) mel = mel[:128,:128] X.append(mel)
      y.append(i)

X = np.array(X) y = np.array(y)

X = X.reshape(X.shape[0],128,128,1)

y = to_categorical(y)


X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2) model = Sequential()
```

```
model.add(Conv2D(32,(3,3),activation='relu',input_shape=(128,128,1)))
 model.add(MaxPooling2D())

 model.add(Conv2D(64,(3,3),activation='relu'))
 model.add(MaxPooling2D())

 model.add(Flatten())

 model.add(Dense(128,activation='relu'))

 model.add(Dense(6,activation='softmax'))

 model.compile(loss="categorical_crossentropy",optimizer="adam",metrics=["accuracy"])

 model.fit(X_train,y_train,epochs=10)  model.save("model.h5")

 print("Model trained successfully")
```

Chapter-6
IMPLEMENTATION OF THE PROJECT

6. IMPLEMENTATION OF THE PROJECT

The implementation of the Deep Learning-Driven Pulmonary Disease Recognition System involves a structured approach to processing lung sound data for accurate disease classification. It begins with preprocessing techniques such as resampling, noise removal, and amplitude normalization to enhance signal quality. The lung sounds are then converted into Mel-Spectrograms, which serve as inputs to a CNN-LSTM model for feature extraction and sequential analysis. The trained model classifies respiratory conditions like Asthma, COPD, Pneumonia, and Bronchiectasis based on the extracted features.

6.1 Setting up the Development Environment:

The development environment is an essential component in building and executing the AI Lung Disease Detection system. It provides the required software tools, programming frameworks, libraries, and runtime platforms necessary for developing, testing, and deploying the application. In this project, the development environment is configured to support machine learning model training, audio signal processing, and web-based user interaction.

The system is implemented using the Python programming language along with several libraries for deep learning, audio processing, and web development. The application also uses the Flask framework to create a web interface that allows users to upload lung sound recordings and view prediction results.

Setting up the development environment involves installing Python, configuring necessary libraries, setting up the project directory structure, and preparing the runtime environment for executing the application.

Before starting the development process, it is necessary to ensure that the system meets the minimum hardware and software requirements.

Hardware Requirements

The following hardware configuration is recommended for smooth operation of the system:

Processor: Intel Core i5 or higher

RAM: Minimum 8 GB

Storage: Minimum 256GB

Audio support: Required for testing lung sound files

These specifications ensure efficient processing of audio data and smooth training of the deep learning model.

Software Requirements

The project requires the following software components:

Operating System: Windows / Linux / macOS

Programming Language: Python 3.x

Web Framework: Flask

Deep Learning Framework: TensorFlow Keras

Audio Processing Library: Librosa

Visualization Libraries: Matplotlib and NumPy

PDF Generation Library: FPDF

Development Environment: Visual Studio Code or any Python IDE

These tools provide the necessary support for developing both the machine learning .

6.1.1 Installation of Python

Python is the primary programming language used in this project. It provides extensive libraries for machine learning, signal processing, and web development**.**

Steps to install Python:

1. Visit the official Python website.
2. Download the latest stable version of Python.
3. Run the installer.
4. Select the option Add Python to PATH during installation.
5. Complete the installation process.

After installation, verify the Python version by executing the following command in the command prompt or terminal:

python --version 3.0

This command displays the installed Python version and confirms that Python has been successfully installed.

6.1.2 Installation of Required Libraries

To implement the AI lung disease detection system, several Python libraries are required. These libraries provide functionality for deep learning, audio analysis, data visualization, and web development.

The required libraries include:
Flask – used to build the web application NumPy – used for numerical computations
Librosa – used for audio signal processing
Matplotlib – used for visualization of waveforms and spectrograms.
TensorFlow – used for training deep learning models
Scikit-learn – used for data splitting and model evaluation.
FPDF – used for generating PDF reports.
These libraries can be installed using the Python package manager

(pip). Execute the following command in the terminal:

pip install flask numpy librosa matplotlib tensorflow scikit-learn fpdf

Once installed, these libraries can be imported into the Python project and used for implementing different functionalities.

6.1.3 Project Folder Structure

After installing the required software and libraries, the next step is organizing the project directory. A well-structured directory helps in managing files efficiently and simplifies the development process.

The project folder structure for the AI Lung Disease Detection system is organized as follows:

Project Root Folder

app.py – Main Flask application file

train_model.py – Deep learning model training

script model.h5 – Trained CNN model

Templates Folder

login.html – User login page signup.html – User registration page

upload.html – Lung sound upload page.

report.html – Prediction results page.

Static Folder

uploads – Stores uploaded lung sound files

waveforms – Stores generated waveform

images spectrograms – Stores Mel spectrogram

imgh8.jpg – Background image used in web interface

Dataset Folder

Healthy – Lung sound recordings of healthy individuals Pneumonia – Pneumonia sound samples

Bronchitis – Bronchitis sound samples Asthma – Asthma sound samples

COPD – COPD sound samples

URTI – Upper respiratory infection sound samples

Users.json – Stores registered user credentials

This structured organization helps separate frontend, backend, and dataset components of the system.

6.1.4 Configuration of Flask Web Framework

Flask is used as the backend framework to create the web interface for the application. It allows users to interact with the system by uploading lung sound recordings and viewing prediction results.

The Flask application is initialized using the following code: from flask import Flask

app = Flask(__name__)

Flask also requires a secret key for managing user sessions securely. app.secret_key = "lung-disease-predictor-secret-key"

This configuration enables the application to handle user login sessions and securely manage interactions.

6.1.5 Setting Up Required Directories

The system requires several directories for storing uploaded files and generated images. These directories are automatically created when the application starts.

The required directories include:
Uploads Folder – stores uploaded lung sound recordings
Waveforms Folder – stores waveform images generated from audio signals
Spectrograms Folder – stores Mel spectrogram visualizations
The directories are created using the following Python code:

```
os.makedirs(UPLOAD_FOLDER,exist_ok=True)
os.makedirs(WAVE_FOLDER,exist_ok=True)
os.makedirs(SPEC_FOLDER, exist_ok=True)
```

This ensures that the necessary folders exist before the system processes audio files.

6.1.6 Running the Application

Once the environment setup is complete, the Flask application can be executed. Run the following command in the terminal:

python app.py

After executing this command, the Flask server starts and the application becomes accessible through the web browser.

The default server address is:

http://127.0.0.1:5000

Users can access the system through this address, log in, upload lung sound recordings, and receive AI-based disease predictions.

6.2 Testing the Setup

Run the Flask Test Server

Step 1 – Run the application

python app.py

Step 2 – Open the browser

http://127.0.0.1:5000

Step 3 – Verify login page loads.

Step 4 – Test signup and login.

Step 5 – Upload lung sound file.

Step 6 – Verify system generates:

- Waveform3
- Spectrogram
- Disease prediction
- PDF report.

6.3 Audio Processing Implementation

Audio processing is one of the most important part of the Lung Disease Detection system. The system works by analyzing lung sound recordings uploaded by the user. These recordings contain valuable information about the breathing patterns of a person. Doctors often identify lung diseases such as asthma, pneumonia, and bronchitis by listening to abnormal sounds like wheezing or crackling in the lungs.

However, raw audio recordings are not always ready to be directly used by a machine learning model. The recordings may contain background noise, silent sections, or different recording formats. Because of these issues, the audio data must be processed before it can be analyzed by the system.

The audio processing in this project is implemented using the Librosa library in Python. Librosa is widely used for audio analysis and provides many functions for loading, processing, and analyzing sound signals.

The main steps involved in the audio processing module are:

1. Loading the audio file
2. Resampling the audio signal
3. Removing silent parts of the recording
4. Normalizing the audio signal
5. Extracting Mel Spectrogram features
6. Generating waveform and spectrogram visualizations

6.3.1 Loading the Audio File

The first step in the audio processing stage is loading the lung sound recording that the user uploads through the web interface. When a user uploads an audio file, it is saved in the system's uploads folder.

The system then reads the audio file using the Librosa library. Librosa converts the audio recording into a numerical signal that the computer can process.

The following code is used to load the audio file:

y, sr = librosa.load(file_path, sr=None)

In this code:

- y represents the audio signal data
- sr represents the sampling rate of the audio file
- file_path indicates the location of the uploaded audio file

After loading the file, the audio signal becomes ready for further processing.

6.3.2 Resampling the Audio Signal

Different audio recordings may have different sampling rates depending on the recording device used. The sampling rate indicates how many sound samples are recorded per second. For machine learning models to work properly, it is important that all audio recordings have the same sampling rate. Therefore, the system converts all audio files to a fixed sampling rate. In this project, the sampling rate is standardized to 4000 Hz. This value is suitable for lung sound analysis because respiratory sounds usually occur in lower frequency ranges.

The following code performs the resampling operation:

y = librosa.resample(y, orig_sr=sr, target_sr=4000) sr = 4000

Resampling ensures that all audio inputs have a consistent format. This helps the deep learning model analyze the data more effectively.

6.3.3 Removing Silence from the Audio

Lung sound recordings often contain silent segments where no breathing sounds are present. These silent sections do not provide useful information for disease detection and may reduce the accuracy of the system.

To solve this problem, the system removes the silent parts of the audio recording. This process helps focus only on the important sections where breathing sounds are present.

The silence removal process is performed using the following code:

```
y, _ = librosa.effects.trim(y)
```

This function automatically detects silent sections at the beginning and end of the audio signal and removes them. As a result, the processed audio contains only meaningful lung sound data.

6.3.4 Normalizing the Audio Signal

The volume of audio recordings may vary depending on the microphone used or the environment in which the recording was made. Some recordings may be too loud, while others may be too soft.

To maintain consistency, the system normalizes the audio signal. Normalization adjusts the amplitude of the signal so that all recordings have a similar sound level.

The following code performs signal normalization:

```
y = librosa.util.normalize(y)
```

Normalization helps the machine learning model analyze the sound patterns more accurately without being affected by differences in recording volume.

6.3.5 Feature Extraction Using Mel Spectrogram

After preprocessing the audio signal, the next step is feature extraction. Feature extraction converts the audio signal into a representation that can be understood by the deep learning model.

In this project, the Mel Spectrogram is used as the main feature representation. A Mel spectrogram shows how different sound frequencies change over time.

Lung diseases often produce unique sound patterns such as:

- Wheezing sounds
- Crackling sounds
- Abnormal breathing rhythms

These patterns can be clearly visualized in a Mel spectrogram. The following code generates the Mel spectrogram:

```
mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
mel_db = librosa.power_to_db(mel)
```

In this code:

- n_mels = 128 represents the number of Mel frequency bands used in the spectrogram.
- power_to_db() converts the spectrogram values into decibels for better visualization.

The Mel spectrogram provides important information about the frequency distribution of lung sounds and helps the deep learning model detect disease patterns.

6.3.6 Waveform Visualization

To help users understand the lung sound signal, the system also generates a waveform representation of the audio recording.

A waveform shows how the amplitude of the sound changes over time. It provides a visual representation of the breathing pattern captured in the recording.

The waveform is generated using the following code:

```
librosa.display.waveshow(y,   sr=sr)
```

The waveform image is saved in the system and later displayed in the report page.

This visualization helps users and doctors observe the breathing signal and identify irregularities.

6.3.7 Spectrogram Visualization

In addition to the waveform, the system also generates a visual representation of the Mel spectrogram.

The spectrogram displays sound frequencies along the vertical axis and time along the horizontal axis. The color intensity represents the strength of the sound frequency.

The spectrogram is generated using the following code:

```
librosa.display.specshow(
   mel_db, sr=sr, x_axis='time', y_axis='mel'
)
```

This visualization helps identify abnormal lung sound patterns such as wheezing and crackles, which are common indicators of respiratory diseases.

The spectrogram images are stored in the system and included in the final prediction report.

6.3.8 Importance of Audio Processing

The audio processing module is very important for the success of the AI Lung Disease Detection system. Without proper preprocessing, the machine learning model may receive noisy or inconsistent data, which can lead to incorrect predictions.

By performing audio preprocessing, the system ensures that the lung sound recordings are clean, consistent, and suitable for analysis.

The benefits of audio processing include:

- Removing unwanted noise and silence
- Standardizing audio signals
- Improving feature extraction
- Increasing prediction accuracy
- Providing visual representations of lung sounds

These improvements help the system detect lung diseases more effectively.

6.4 Explanation of Modules

The proposed system for Deep Learning-Driven Phonopneumographic Analysis for Pulmonary Disease Recognition is divided into several modules. Each module performs a specific task in the system to ensure accurate analysis and classification of respiratory diseases based on lung sound recordings.

The modular architecture helps in organizing the system efficiently and allows each component to perform its function independently while working together as a complete pipeline. The system mainly consists of the following modules:

1. Preprocessing Module
2. Data Augmentation Module
3. Time-Frequency Representation (TFR) Extraction Module

6.4.1 Preprocessing Module

The preprocessing module is the first stage of the proposed system. Raw lung sound recordings obtained from electronic stethoscopes or respiratory sound databases often contain noise, inconsistent sampling rates, and amplitude variations. These issues can negatively affect the performance of the classification model. Therefore, preprocessing is necessary to prepare the audio signals before further analysis.

The main objective of the preprocessing module is to improve the quality of lung sound signals and ensure that the input data is standardized for the deep learning model.

The preprocessing stage consists of several steps including resampling, temporal segmentation, noise removal, and amplitude normalization.

Resampling

Lung sound recordings may be collected at different sampling frequencies depending on the recording device. To maintain consistency across the dataset, the audio signals are resampled to a standard sampling rate of 4 kHz.

Resampling ensures that all recordings have the same frequency resolution and prevents inconsistencies during feature extraction.

For example, if one audio file is recorded at 8 kHz and another at 44.1 kHz, the system converts both signals to 4 kHz before processing them further.

Standardizing the sampling rate also reduces computational complexity and speeds up the processing stage.

Temporal Snippet Generation

Lung sound recordings are often long and may contain multiple breathing cycles. Processing the entire recording at once may introduce unnecessary information and reduce model efficiency.

To solve this problem, the long audio signals are divided into smaller segments called temporal snippets.

Each snippet represents a short interval of the lung sound signal. These snippets allow the system to focus on smaller portions of the signal and capture meaningful respiratory patterns.

This segmentation process improves the learning capability of the deep learning model and increases the number of training samples available in the dataset.

Temporal segmentation also helps detect abnormal breathing sounds such as wheezes and crackles more effectively.

Noise Removal using Discrete Fourier Transform

Lung sound recordings often contain background noise from the environment or recording equipment. Noise can interfere with the accurate analysis of respiratory signals.

To remove unwanted noise, Discrete Fourier Transform (DFT) based filtering is applied to the audio signals.

DFT converts the audio signal from the time domain to the frequency domain. In the frequency domain, unwanted low-frequency components and baseline wander can be easily identified and removed.

By filtering these unwanted frequencies, the system improves the clarity of the lung sound recordings.

Noise removal is particularly important when detecting subtle respiratory sounds such as crackles and wheezes.

Amplitude Normalization

Another important step in preprocessing is amplitude normalization.

Different lung sound recordings may have different volume levels depending on the recording device and environment. If these differences are not corrected, the deep learning model may misinterpret loud sounds as important features.

Amplitude normalization ensures that all recordings have consistent signal intensity levels.

In this project, the Librosa library is used to normalize the amplitude of lung sound signals.

Normalization improves the stability and accuracy of the classification model.

6.4.2 Data Augmentation Module

One of the major challenges in machine learning applications is the availability of large datasets. Deep learning models require large amounts of data to learn meaningful patterns and generalize well to new inputs.

However, respiratory sound datasets are often limited in size. To overcome this limitation, data augmentation techniques are applied to artificially increase the size of the dataset.

Data augmentation generates new training samples by applying small transformations to the original audio signals.

These transformations help the model learn more robust features and reduce the risk of overfitting.

Adding Noise

Background noise is added to the original lung sound recordings to simulate real-world recording conditions.

Adding noise makes the model more robust and capable of handling noisy input signals.

This technique ensures that the model performs well even when the input recordings contain environmental noise.

Time Shifting

Time shifting involves slightly shifting the audio signal forward or backward along the time axis.

This transformation creates new variations of the original recording without changing its characteristics.

Time shifting helps the model learn that the exact position of a sound in time is not always important for classification.

Pitch Variation

Pitch variation slightly changes the frequency characteristics of the audio signal.

This technique generates new training samples with different pitch levels while preserving the overall structure of the lung sound.

Pitch variation increases dataset diversity and helps the model learn generalized features.

Speed Variation

Speed variation changes the playback speed of the audio signal without affecting its pitch.

This technique simulates different breathing speeds and adds more diversity to the dataset.

By applying these augmentation techniques, the dataset becomes larger and more balanced.

6.4.3 Time-Frequency Representation (TFR) Extraction Module

The next module in the system is the Time-Frequency Representation (TFR) Extraction Module.

Lung sound signals contain complex patterns that vary over time and frequency. Therefore, analyzing these signals in the time domain alone is not sufficient.

To capture both temporal and spectral characteristics of the audio signals, the system converts them into time-frequency representations.

The most commonly used representation for audio analysis is the Mel Spectrogram.

Spectrogram Generation

A spectrogram is a visual representation of how the frequency components of a signal change over time.

The spectrogram is generated by applying the Short-Time Fourier Transform (STFT) to the audio signal.

STFT divides the signal into small overlapping windows and computes the frequency spectrum for each window.

Mel Spectrogram

In this project, the spectrogram is further processed to generate a Mel spectrogram.

The Mel scale is designed to mimic the way humans perceive sound frequencies.

Lower frequencies are represented with greater resolution, while higher frequencies are compressed.

This representation is particularly useful for analyzing respiratory sounds because it captures subtle frequency variations associated with lung diseases.

Mel spectrograms provide a compact and informative representation of lung sound signals.

6.4.4 CNN-LSTM Classification Module

The final module of the system is the CNN-LSTM deep learning model, which performs the classification of respiratory diseases.

This model combines the advantages of Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks.

Convolutional Neural Network (CNN)

CNNs are widely used for image analysis tasks.

In this project, Mel spectrograms are treated as images and fed into the CNN layers.

The CNN layers perform convolution operations to detect important features such as patterns, edges, and textures in the spectrogram images.

These features represent the acoustic characteristics of lung sounds.

Multiple convolutional layers are used to extract hierarchical features from the spectrograms.

Long Short-Term Memory (LSTM)

LSTM networks are a type of recurrent neural network designed to handle sequential data.

Lung sound signals have temporal dependencies because breathing patterns occur over time.

The LSTM layers capture these temporal relationships and help the model understand how sound patterns evolve during the breathing cycle.

By combining CNN and LSTM, the system can analyze both spatial and temporal characteristics of lung sound signals.

Output Layer

The final layer of the model is a SoftMax classification layer.

This layer outputs the probability of each respiratory disease class.

The model predicts the most likely disease category based on the extracted features.

The diseases classified by the system include:

- Asthma
- Bronchitis
- Pneumonia
- Bronchiolitis
- COPD
- Lower Respiratory Tract Infection (LRTI)

After training the model, it is used to predict respiratory diseases based on new lung sound recordings.

When a user uploads a lung sound recording, the system performs the following steps:

1. Preprocess the audio signal
2. Generate Mel spectrogram features
3. Feed the features into the trained CNN-LSTM model
4. Predict the respiratory disease category

The system then displays the predicted disease along with precautionary measures for the patient.

Chapter-7
TESTING

7.1 Testing

Software testing is the process of evaluating and verifying that a software application performs according to specified requirements. The AI Lung Disease Detection System was tested to ensure reliable analysis of lung sound recordings, accurate disease prediction, and proper generation of diagnostic reports.

The benefits of testing include:

1. Preventing functional errors
2. Ensuring accurate disease prediction
3. Improving system reliability and performance

7.2 Types of Testing

1. Manual Testing

Manual testing involves executing test cases manually without automated tools. In this project, manual testing was performed to validate user interactions, file uploads, system responses, and report generation.

Manual testing ensured that:

- Users can create accounts and log in successfully
- Lung sound files can be uploaded correctly
- AI analysis results are displayed properly
- PDF reports are generated and downloaded
- Invalid inputs are handled appropriately

Manual testing helped identify interface issues and ensured the system behaves correctly under normal usage conditions.

2. Black Box Testing

Black box testing evaluates system functionality without examining internal code structure. The system was tested by providing different lung sound inputs and verifying outputs such as detected disease, severity level, visualizations, and recommendations.

Key validations included:

- Correct disease classification based on input audio
- Accurate severity determination
- Proper display of waveform and spectrogram
- Appropriate AI recommendations for each disease

Only inputs and outputs were tested to confirm expected behavior.

3. Integration Testing

Integration testing verifies that individual modules work together correctly as a complete system.

In this project, integration testing ensured seamless interaction between:

- User Authentication Module
- Audio Upload Module
- Audio Preprocessing Module
- Feature Extraction Module
- Disease Prediction Module
- Visualization Module
- PDF Report Generation Module

For example, the uploaded audio file is preprocessed using resampling and normalization, then analyzed to extract features, classify disease, generate graphs,

4. System Testing

System testing evaluates the entire application as a whole to ensure it meets functional and performance requirements.

This testing phase focused on:

- End-to-end workflow from login to report download
- Accuracy of disease prediction
- Visualization correctness
- System stability during file processing
- Usability and responsiveness Key Aspects of System Testing

1) Functional Testing:

Validates core features such as login, signup, audio upload, disease detection, visualization, and report generation.

2) Performance Testing:

Checks system response time when processing audio files and generating graphs and PDF reports.

7.3 Test Cases

Table 7.3.1: Test Case 1

Test Case	Expected Output	Actual Output
Does the system provide a user login portal?	When the homepage loads, it should display a login form.	The homepage displays a login form.

Table 7.3.2: Test Case 2

Test Case	Expected Output	Actual Output
What happens when a user enters valid credentials?	The system should authenticate and redirect to the upload page.	The system authenticates and redirects to the upload page.

Table 7.3.3: Test Case 3

Test Case	Expected Output	Actual Output
Can a new user create an account?	The system should register the user and allow login.	The system registers the user and allows login.

Table 7.3.4: Test Case 4

Test Case	Expected Output	Actual Output
Can the user upload a lung sound file?	The system should accept audio input and process it.	The system should accept audio input and process it.

Table 7.3.5: Test Case 5

Test Case	Expected Output	Actual Output
Does the system analyze the lung sound?	The system should detect disease and severity level.	The system displays detected disease and severity level.

Table 7.3.6: Test Case 6

Test Case	Expected Output	Actual Output
Are waveform and spectrogram displayed?	The system should generate and display visual graphs.	The system displays waveform and Mel-spectrogram graphs.

Table 7.3.7: Test Case 7

Test Case	Expected Output	Actual Output
Does the system provide AI recommendations?	The system should display precautions based on disease.	The system displays relevant recommendations.

Table 7.3.8: Test Case 8

Test Case	Expected Output	Actual Output
Can the user download the report?	The system should generate a PDF report for download.	The system generates and downloads the PDF report successfully.

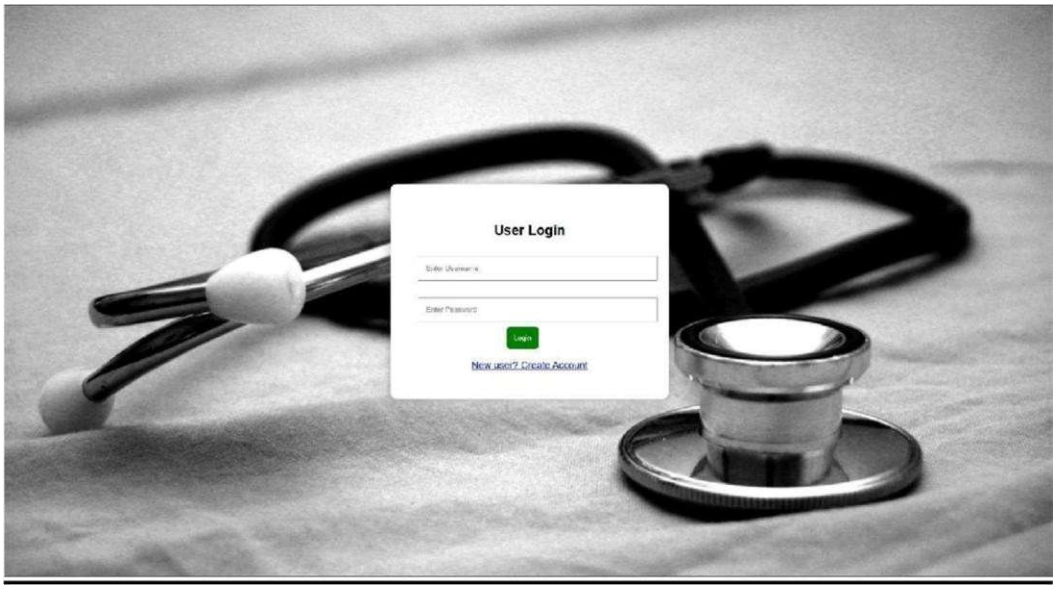
Chapter-8
RESULTS

8.1 Results

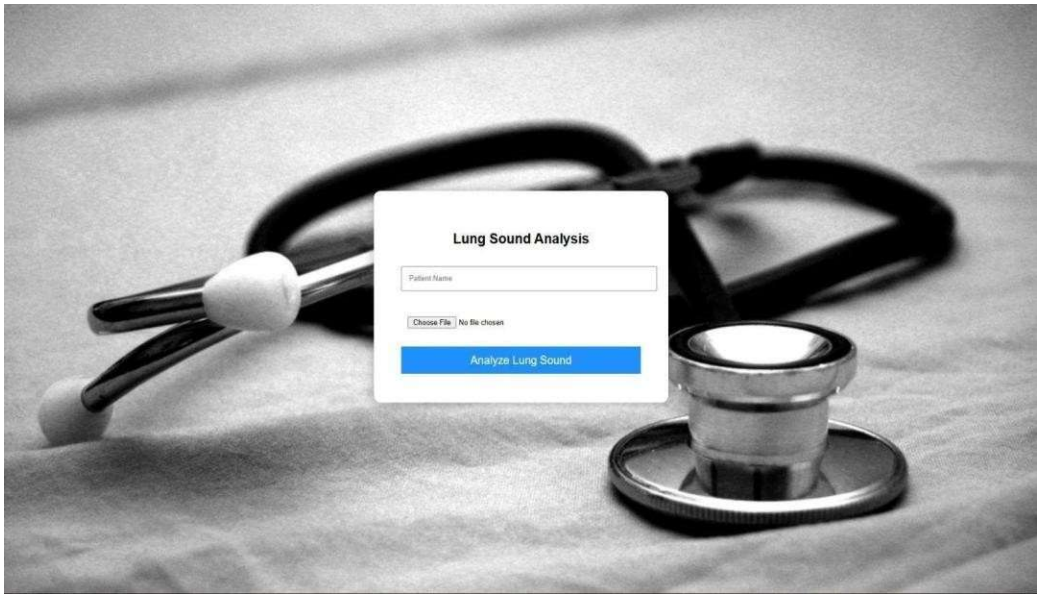
Create User Account



User login page



Upload Lung Sound Recording



AI Lung Disease Detection Report

AI Lung Disease Detection Report

Patient Name: RAMU

Detected Disease: Bronchitis

Severity Level: Mild

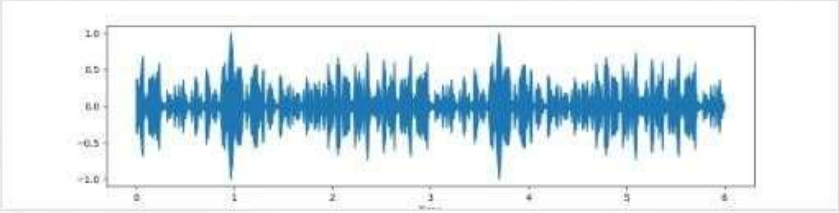
AI Analysis

Mucus congestion and airway irritation suggest bronchitis.

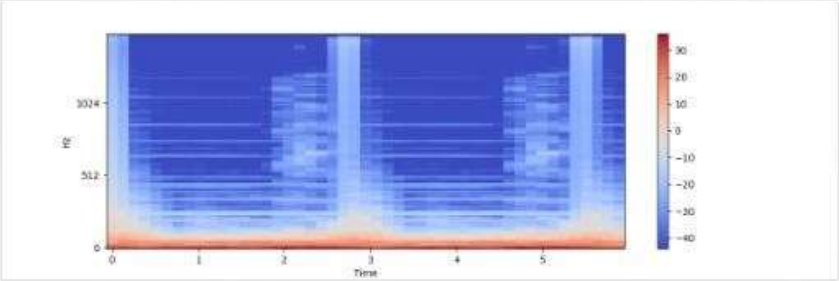
Normal Lung Sound Reference

Rhonchi due to mucus buildup.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Drink warm fluids.
- Inhale steam.
- Avoid smoking.
- Take adequate rest.
- Use humidifier.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

Prediction Result Page:

AI Lung Disease Detection Report

Patient Name: User

Detected Disease: Pneumonia

Severity Level: Moderate

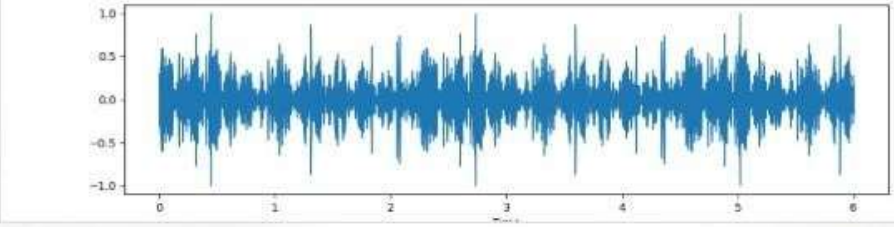
AI Analysis

Crackling respiratory sounds suggest pneumonia.

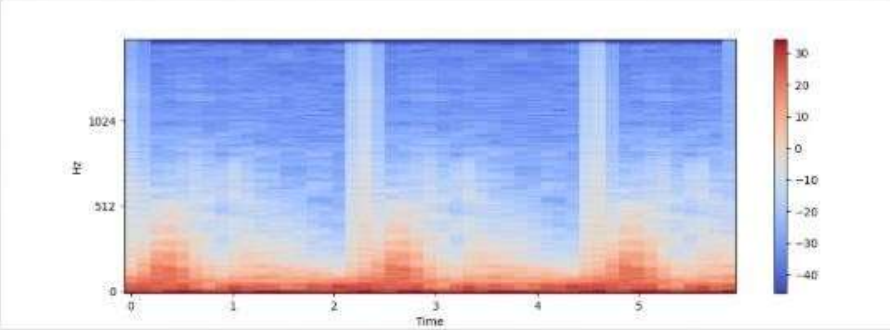
Normal Lung Sound Reference

Crackles from fluid in lungs.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Rest and hydration.
- Controlled breathing exercises.
- Follow medical prescription.
- Avoid cold exposure.
- Consult doctor if needed.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

AI Lung Disease Detection Report

Patient Name: RAMU

Detected Disease: COPD

Severity Level: Moderate

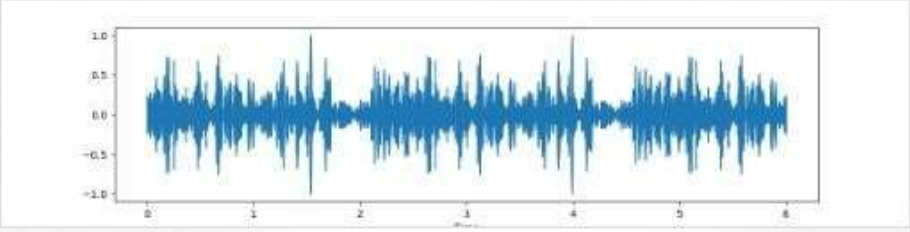
AI Analysis

Airflow limitation patterns suggest COPD.

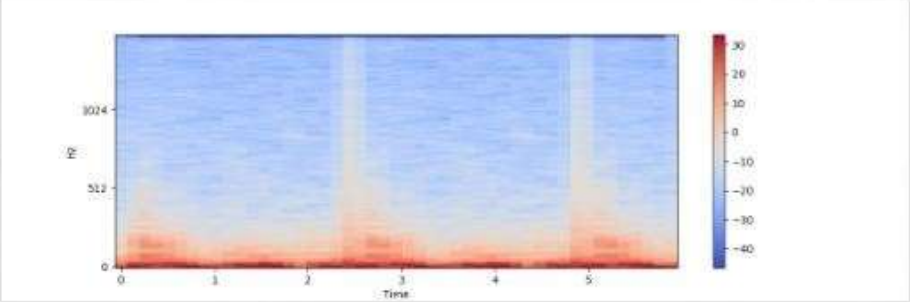
Normal Lung Sound Reference

Airflow obstruction pattern.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Pursed lip breathing.
- Avoid smoking.
- Maintain healthy weight.
- Pulmonary exercises.
- Improve indoor air.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

Model Accuracy Comparsion:

AI Lung Disease Detection Report

Patient Name: nani

Detected Disease: Healthy

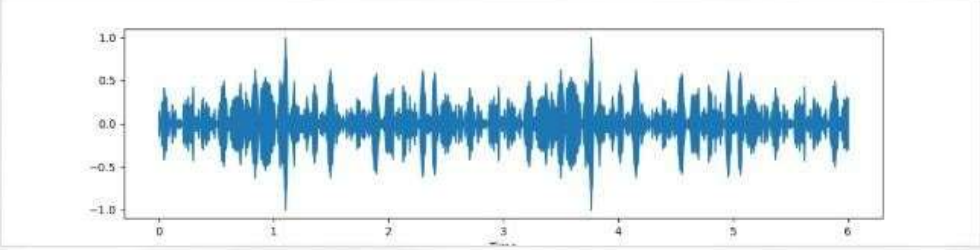
AI Analysis

No abnormal respiratory patterns detected.

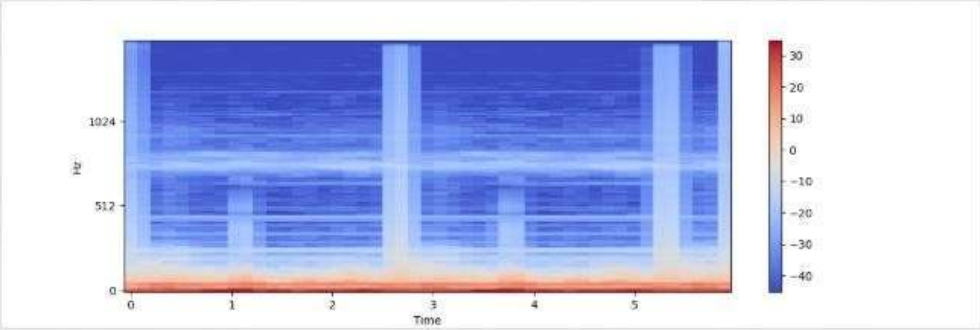
Normal Lung Sound Reference

Normal smooth airflow lung sounds.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Regular exercise.
- Deep breathing.
- Clean air environment.
- Avoid smoking.
- Balanced diet.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

AI Lung Disease Detection Report

Patient Name: RAJU

Detected Disease: Asthma

Severity Level: Moderate

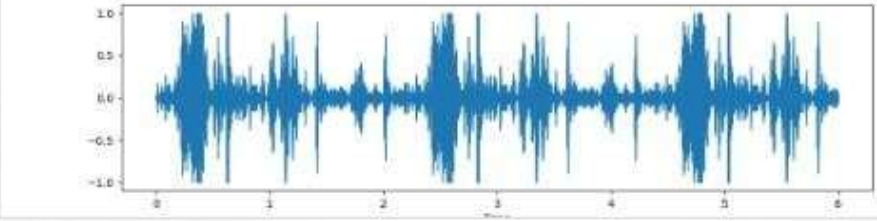
AI Analysis

Airway narrowing and wheezing patterns suggest asthma.

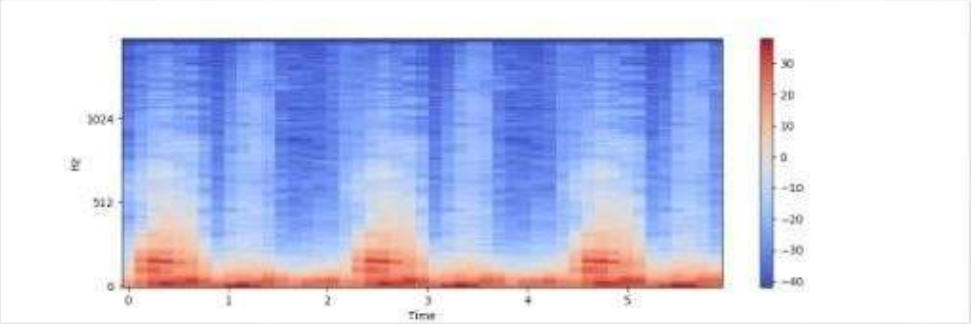
Normal Lung Sound Reference

Wheezing sound from airway narrowing.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Practice diaphragmatic breathing.
- Avoid allergens.
- Light aerobic exercise.
- Use inhaler medication.
- Maintain clean air.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

AI Lung Disease Detection Report

Patient Name: RAMU

Detected Disease: COPD

Severity Level: Moderate

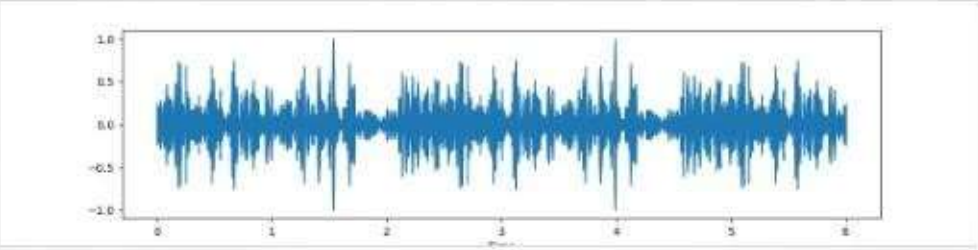
AI Analysis

Airflow limitation patterns suggest COPD.

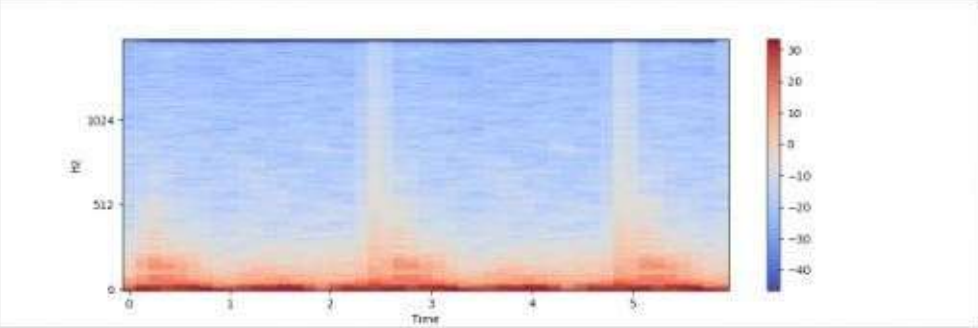
Normal Lung Sound Reference

Airflow obstruction pattern.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Pursed lip breathing.
- Avoid smoking.
- Maintain healthy weight.
- Pulmonary exercises.
- Improve indoor air.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

Final Output Prediction Page:

AI Lung Disease Detection Report

Patient Name: RAMU

Detected Disease: URTI

Severity Level: Moderate

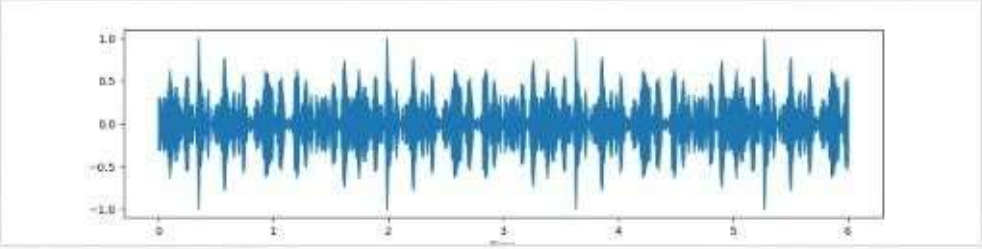
AI Analysis

Sound patterns indicate upper respiratory infection.

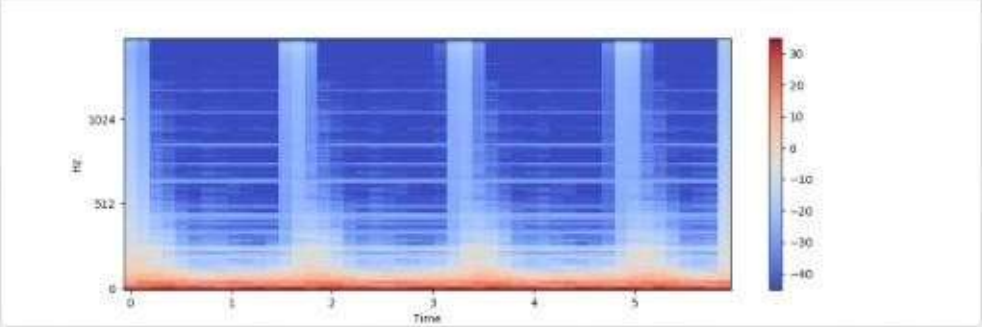
Normal Lung Sound Reference

Upper airway turbulence.

Lung Sound Waveform



Mel Spectrogram



AI Recommendations

- Warm fluids.
- Steam inhalation.
- Vitamin rich diet.
- Proper rest.
- Avoid sudden temperature change.

This prediction is generated using an AI respiratory sound analysis system. It is intended for research and educational purposes only and should not replace professional medical diagnosis.

Download PDF Report

Chapter-9
CONCLUSION AND FUTURESCOPE

9. CONCLUSION AND FUTURESCOPE

9.1 Conclusion:

The Deep Learning-Driven Phonopneumographic Analysis for Pulmonary Disease Recognition system provides an advalnced, AI-powered approach to detecting lung diseases through automated sound analysis. By leveraging Deep Learning techniques such as CNN-LSTM models, this system efficiently classifies lung sounds into various disease categories, ensuring early detection and diagnosis. The integration of DFT (Discrete Fourier Transform) and Mel-Spectrograms enhances feature extraction, improving the model's ability to distinguish between different respiratory conditions. This method reduces manual diagnostic errors, making it a valuable tool for healthcare professionals and researchers.

The system offers a user-friendly interface, allowing medical professionals to upload lung sound recordings, process data, and receive real-time classification results. The automated prediction model ensures high accuracy and reliability, reducing the need for extensive manual interpretation of lung sounds. Additionally, the system provides a detailed classification report, which can be printed or downloaded for further analysis. The use of AI and machine learning in respiratory disease detection not only enhances efficiency but also contributes to the advancement of digital healthcare solutions.

In conclusion, this project demonstrates the potential of deep learning in medical diagnostics, particularly in pulmonary disease recognition. With continuous improvements and further model training on larger datasets, the system can become an even more precise and effective tool for assisting healthcare providers. Future enhancements may include real-time monitoring, integration with telemedicine platforms, and expanded disease classification capabilities. By bridging the gap between AI and medical diagnostics, this project contributes to early disease detection, better patient outcomes, and improved healthcare accessibility worldwide.

9.2 Future Scope:

The Deep Learning-Driven Phonopneumographic Analysis for Pulmonary Disease Recognition system has significant potential for future enhancements and advancements in AI-driven healthcare. One of the key areas for improvement is real- time monitoring, where the system can be integrated with wearable devices or mobile applications to continuously track lung health. This would enable early detection of respiratory issues and provide timely alerts to patients and healthcare providers. Additionally, increasing the dataset size by incorporating diverse lung sound recordings from different demographics can further improve model accuracy and generalization.

Another important future enhancement is integration with telemedicine platforms to facilitate remote diagnosis and consultation. By enabling cloud-based deployment, doctors can analyze lung sounds from patients worldwide, making healthcare more accessible, especially in rural and underprivileged areas. The system can also be expanded to classify a wider range of respiratory diseases, including rare conditions, by training on larger and more diverse datasets. Additionally, implementing explainable AI (XAI) techniques can enhance the system's transparency, helping medical professionals understand and trust AI-generated predictions.

Looking ahead, further optimization of the CNN-LSTM model can lead to faster and more efficient processing, reducing computational requirements and enabling real- time classification on low-power devices like smartphones and IoT-based sensors. Collaborations with medical institutions and researchers can also contribute to refining the system by incorporating expert-validated lung sound datasets. With continuous advancements in deep learning and AI technologies, this project has the potential to revolutionize pulmonary disease diagnosis, making healthcare more accurate, efficient, and accessible worldwide.

Chapter- 10

BILBLIOGRAPHY

10. BIBLIOGRAPHY

Reference Books:

1. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Tom M. Mitchell, *Machine Learning*, McGraw-Hill Education, 1997.
3. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Al Sweigart, *Automate the Boring Stuff with Python*, No Starch Press, 2015.
5. Textbook of Pulmonary and Critical Care Medicine – S.K. Jindal, 2017.
6. The Principles of Pulmonary Diagnosis: Critical Concepts and a Structured Approach Ashfaq Hasan, 2025.
7. Principles of Pulmonary Medicine (8th Edition) – Steven E. Weinberger, Barbara A. Cockrill, Jess Mandel, 2023.
8. Disease Prediction using Machine Learning, Deep Learning and Data Analytics – Geeta Rani, Vijaypal Singh Dhaka, Pradeep Kumar Tiwari, 2024

E-References:

1. https://react.dev/
2. https://nextjs.org/docs
3. https://firebase.google.com/docs
4. https://tailwindcss.com/docs
5. https://platform.openai.com/docs
6. https://ai.google.dev/
7. https://www.geeksforgeeks.org/
8. https://stackoverflow.com/
9. https://www.w3schools.com/
10. https://developer.mozilla.org/
11. https://www.tensorflow.org/
12. https://librosa.org/doc/latest/index.html
13. https://flask.palletsprojects.com/

Research Papers:

1. “Respiratory Sound Classification Using Deep Learning” – IEEE Xplore.

2. “Automated Detection of Lung Diseases Using CNN” – Springer Journal.

3. “Lung Sound Analysis for Disease Detection Using Machine Learning” – International Journal of Computer Applications.

4. “Deep Learning for Healthcare: Review and Opportunities” – IEEE Access.

5. “Audio-Based Diagnosis of Respiratory Diseases Using MFCC and CNN” – Elsevier Journal.